Context-Aware Agentic Orchestration for Adaptive Edge–Cloud AI Inference Using Reinforcement Learning

Name: Najish Mahmud

Student ID: 2514376

MSc Project Report

Unit Title: MSC Project - Artificial Intelligence

Unit Coordinator: Dr Renxi Qiu

Contents

1 Abstract

Existing cascade systems like BranchyNet, MSDNet, EdgeBERT, RouteLLM use hardcoded confidence thresholds or hardware timers to route requests. They do not use live metrics from infrastructure state. This study explores a context-aware agentic gateway which uses an edge context protocol, a dynamic Medallion pipeline, MCP skills, PPO agents, bidirectional loop and a policy orchestration layer to make dynamic routing decisions. Here the PPO is trained offline in a Gymnasium simulation with domain randomization which generates a frozen policy. This policy is deployed to the PPO agent to make routing decisions. For steady and bursty types of traffic, the proposed system reduces cloud escalations from 100%, i.e. static-threshold baselines, down to 19.5% under both, eliminating SLA violations under bursty load entirely (down from 65–66% for static thresholds) at a small, consistent accuracy cost (about one percentage point). This result required two rounds of diagnosis and correction during evaluation: the first trained policy was undertrained and was silently governed by its own deterministic safety fallback rather than its learned action, and once fixed, an unclamped-latency reward term was found to give the policy no gradient against escalating once an SLA was already violated, which it disproportionately did under sustained network degradation. Both are reported as part of the evaluation, not concealed. The system was compared against five baselines and eight literature baselines across three traffic scenarios, plus a seven-run ablation, including a governance-layer ablation comparing a shared frozen policy against a policy orchestration layer that can reuse, fine-tune, or train a new policy per client and, since a small local LLM was added as an oversight mechanism, escalate its own decision when warranted.

2 Introduction

2.1 Problem Statement

Consider a confidence-only cascade router like that of RouteLLM (Ong et al., 2024) where a classifier is reading a request and deciding whether a cheap or expensive model should answer it. It is a choice made by comparing preference data the models themselves produce against the input. The edge device answering that request is under continual CPU and thermal stress. Guo et al. (2017) have shown that this kind of stress causes present-day neural networks to produce wrong results: a model's softmax output stays confident in a particular result while the prediction underneath it degrades. A router that bases its decision solely on the confidence value therefore cannot tell apart an easy input from a difficult one the device has answered wrongly because its confidence signal has drifted. Such a router returns a confidently wrong answer to the edge, when the true cause of the error was the load on the device rather than the difficulty of the input. This is the first of two failure modes this project addresses. Failure A is that the routing decision never sees the operational health of the edge device at the moment of inference. BranchyNet (Teerapittayanon et al., 2016) and MSDNet (Huang et al., 2018) share the same limitation from the other side: both rely on either a confidence or entropy threshold, or a computational budget fixed at design time, and neither reads the real-time condition of the infrastructure it runs on. EdgeBERT (Tambe et al., 2021) goes further by reacting to the state of the hardware, but limits itself to its own chip's entropy/voltage-frequency envelope.

A second failure sits on top of the first. Present-day cascade systems run the same fixed enrichment pipeline over every request, whatever state the edge is in, and no outcome ever travels back to it. Whether the SLA was met, or whether the cloud would have been the more accurate choice, changes nothing about how the next request from that device is handled, even after repeated deadline misses. Failure B is the absence, across every system surveyed in Section 3, of both a dynamic pipeline that measures differently according to the current edge situation, and a return path that lets an outcome correct how the edge is read.

The whole scenario is what motivates the research question of this dissertation: what is gained by replacing static passive thresholds with a truly active orchestration system that can see live data about the environment around it, reason over it, take actions, and consider the consequences of those actions?

2.2 Aims and Objectives

1. Survey the landscape of cascade systems, edge-cloud offloading and agent AI architectures (Years 2020 to 2026). Locate the artifact relative to RouteLLM, a genetic algorithm NSGA-II router Splitwise GreenServ and PROTEUS.
2. Model the decision to escalate as a Markov Decision Problem where the state is up to 13 variables (which can vary via the dynamic Medallion pipeline including per resource stress, etc.), the actions

are { route to edge, escalate to cloud, fetch extended context }, and the reward function is designed to balance accuracy, latency, SLA compliance, and energy.

3. Implement the dynamic Medallion pipeline: register in a registry of bronze metrics, condition-based enrichment in silver, and gold masked state vector.

4. Carry out the Edge Context Protocol through self-reports from the edge and backward loops.

5. Use a domain-randomized Gymnasium simulation to train PPO offline. Thereafter the policy is frozen and deployed as its own service reachable over HTTP, live-tested twice: as a plain-Docker multi-region GCP deployment, and as a genuine 3-region Kubernetes (GKE) deployment with the current policy (Section 5). Neither path is a literal sidecar (one PPO process co-located inside another container). See Section 5's Deployment subsection for the honest distinction.

6. Provide a dynamic registration and list_tools() method of the MCP skill interface.

7. Performance test against 5 baseline approaches and evaluate on three different traffic conditions. In addition, conduct a multi-run ablation study (all seven runs of the plan executed, Section 7.1).

2.3 Contributions

1. Edge as an actor. Per the paper, the Edge Context Protocol is a new way of designing edge devices in which the devices report their calibration state actively rather than just provide predictions, which closes failure A, a whole type of error that BranchyNet MSDNet EdgeBERT and RouteLLM fail to discover.

2. Medallion Pipeline with Dynamic Components. A pipeline which allows a pipeline to self-modify its pipeline per request with data from the edge, generating state vectors with variable dimensions instead of a fixed dimension.

3. SLA remaining and predicted RTT as per-request signals at routing level. Every routing decision sees the current request's own SLA remaining and predicted network RTT, not an aggregate cluster-level budget, a granularity missing from Splitwise (cluster/phase-level scheduling) and every cascade baseline studied. This is a soft, learned preference (the reward function penalises an SLA violation directly) rather than a hard per-request gate that blocks a doomed escalation before it happens. Section 5 states this distinction plainly, including a Silver-layer signal computed for that harder guarantee but not yet wired into any decision path.

4. MCP Skills Interface at Inference Latency. A dynamic per-request tool discovery and registration pattern that lets a synchronous ML service make use of infrastructure signals well within the per-request perception budget.

5. Two-way agent loop. Escalation outcomes feed back into the edge's own context state within the request lifecycle itself, not only through a slow offline retraining loop that doesn't exist in this design. Together with the dynamic pipeline above, this closes failure B.

6. Policy Orchestration Layer. Policy orchestration at a fleet level is a layer of governance of the fleet, which for every client, decides: re-use, fine-tune or train a new policy. All these choices are supported via the task-capacity promotion test (Bossens and Sobey, 2021). Every choice is permanently logged in a way where it can be verified later.

3 Literature Survey

3.1 Confidence- and budget-only cascades

BranchyNet (Teerapittayanon et al., 2016) installs side-branch classifiers on a deep network and allows a sample to leave early as soon as softmax entropy falls under a learned threshold. This is an idea that has proven very fruitful to the point that early features are usually very discriminative for easy inputs and the design is still the basis of most cascading systems that have been developed later on. The decision to exit the network remains a function of the input only: BranchyNet has neither a cloud queue depth nor a network round-trip time (RTT) signal nor an SLA budget nor a health indicator (of a running device) so it simply cannot find out that the next layer is currently overloaded.

MSDNet (Huang et al., 2018) takes the idea further by enabling both "anytime" and "budgeted-batch" classifications - i.e. spending a fixed computational budget differently on easy and hardest parts of inputs within one model, So getting rid of the necessity of having separated networks per computation budget which had been the case before. Although the budget itself is a design-time only constant, not a measurement capturing the current infrastructure state. It cannot see the growth of the cloud queue or the RTT spike just one second ago.

EdgeBERT (Tambe et al., 2021) is, by far, the most similar previously developed system to infrastructure-aware routing and it should be recognised for it: The use of entropy for early exit prediction is linked to dynamic power scaling (DVFS) on per-sentence basis which adapts the power capabilities of the chip to the difficulty level of the input in real time, supported by actual accelerator hardware. But its state is solely restricted to the device's own chip - it has no interface connecting it to the external signals of infrastructures such as the status of the cloud queue or the health of the network, because the closure of that loop has never been one of its problems.

SPINN (Laskaridis et al., 2020) splits a single deep network across the edge device and the cloud, running the early layers on-device and the remainder on a cloud partner, with the exit-versus-offload decision conditioned on both softmax entropy and a measured network connectivity signal rather than entropy alone. This is the closest any surveyed system comes to combining an on-device signal with a network-side one, and it deserves the same critical treatment as its neighbours. The connectivity signal SPINN reads is a point measurement of link quality taken at split time, not a live cloud queue depth or a per-request SLA deadline. It therefore still cannot tell a saturated cloud endpoint from a lightly-loaded one. Its split point is also fixed once the network is partitioned, rather than adapted from an outcome fed back after the request completes. The same limitation BranchyNet and MSDNet share.

3.2 Learned, single-signal routers

RouteLLM (Ong et al., 2024) teaches a router purely from human preference data to distinguish between a stronger and a weaker LLM by a few data points and the system shows more than a 2x cost reduction while also generalizing well to model pairs the router has not seen in training. That is a significant result, being the closest deployed comparison for a learned router. But the only context RouteLLM knows is the prompt itself. It is not only unfamiliar but has never come across the concept of serving-system load. In fact, from a RouteLLM router's viewpoint, a flooded cloud endpoint and a spare one would look the same and be handled in similar ways.

The NSGA-II router (Yu et al., 2025) goes much further towards multi-objective, infrastructure-aware routing. It uses an evolutionary algorithm to manage and balance quality, speed, and the cost of a response through heterogeneous edge and cloud nodes. It also keeps up with the diversity of requests and nodes. The problem is that even though the set of optimization objectives is the correct one, they are only used offline to generate a Pareto-optimal policy fixed ahead of requests, unlike a router, which must react to changes in live queue or RTT during serving to make decisions that are request- or -queue-aware. The PROTEUS model (Bhatti et al., 2026) is the closest in approach to the system built in this project. It's a Lagrangian-constrained RL policy that accepts a runtime accuracy target as input, routes over a series of LLMs, while enforcing a minimum level of accuracy.

The study demonstrated how the model could achieve accuracy up to 1.3%-up to 4.6% close to an oracle with cost savings of up to 89.8% vis-a-vis the best fixed model, a very impressive outcome of RL applied to routing. On the downside, the state space of PROTEUS does not contain any information about queues, latency energy etc. The algorithm is trained to trade off precision against cost without considering such infrastructure changes upon which the routing in this paper is based.

GreenServ (Ziller et al., 2026) is the closest of the three learned routers on the energy dimension: it routes each query to whichever model in a heterogeneous pool of large language models has historically balanced accuracy and energy use best for a query with that profile, using a multi-armed bandit that keeps learning online rather than a policy fixed once at training time. That online adaptivity is exactly what BranchyNet, MSDNet and EdgeBERT lack. But the context GreenServ conditions on is entirely query side (task type, semantic cluster, text complexity) plus its own model pool's observed accuracy/energy history. Like RouteLLM, it has no channel for cloud queue depth, network RTT, per-request SLA remaining, or the operational health of the device issuing the request, so two structurally identical requests are routed identically by GreenServ regardless of whether the infrastructure behind them is idle or saturated at that moment.

3.3 Cluster-aware serving systems

Patel et al. (2024), in their paper on Splitwise, highlight that it is the most operationally-aware system reviewed in this chapter so far: the system performs an operation-wise (compute v. memory bottleneck) phase separation and runs each phase on separate machines that specialize in that phase. Splitwise explains the differences in the

phase capabilities (throughput power etc.) so well because it is a real case of heterogeneous hardware in servers. Scheduling is at the level of clusters and phases, but routing decisions are not accompanied by a per-request SLA deadline check.

3.4 Commercial and open-source landscape

The nine academic systems above are not the only relevant comparison: a mature market of "LLM gateway" products already solves a version of the same routing problem in production, and a fair positioning has to account for what they actually do, not only what research papers propose. Two of the most widely deployed were checked against their own documentation rather than their marketing pages.

LiteLLM (an open-source proxy self-hosted in front of 100+ model providers) documents six routing strategies for its `Router` class: simple-shuffle (weighted pick, the recommended production default), latency-based (lowest recent response time), usage-based (lowest current token throughput), least-busy (fewest in-flight calls), cost-based, and a pluggable custom-strategy interface. It also has a cooldown mechanism that temporarily stops routing to a deployment after a run of rate-limit or failure responses, and a latency cache that records recent response times per deployment. This is a reactive system: it does respond to measured latency and failures. Every one of its strategies is nonetheless a fixed rule evaluated against a rolling window of recent responses, so none of them is a learned policy. None reads a self-reported edge-device health or calibration state either, since LiteLLM routes between cloud model endpoints rather than edge devices, leaving no analogous self-report to read. Nor does any strategy enforce a per-request deadline before dispatch: a request goes to whichever deployment the current rule favours, and if that deployment is slow, the outcome is observed only after the fact.

Portkey's AI Gateway documents its load-balancing as static weighted distribution across configured providers or models. Traffic is split according to weights the operator sets (e.g. for cost optimisation or gradual migration testing), normalised to sum to 100%, not according to a live queue, latency, or health signal measured at request time. Portkey's own documentation for this feature makes no mention of reinforcement learning, closed-loop adaptation from outcomes, or per-request SLA enforcement. Those remain configuration choices made by the operator up front, not decisions the gateway itself learns or adapts.

The wider landscape (OpenRouter, Envoy AI Gateway, Requesty, and Martian's now-discontinued standalone "intelligence router") occupies the same territory: cost- and latency-aware routing across a pool of model providers, marketed as adaptive but, on the evidence available in each product's own documentation, implemented as configured rules or a pre-trained model-performance predictor rather than a system that learns from live infrastructure telemetry and its own routing outcomes. None of them is edge-cloud specific in the sense this project is (they route between cloud model providers, not between a resource-constrained edge device and a cloud fallback) so the edge-health self-report, the deadman's switch, and the bidirectional trust loop described in Section 5 do not have a commercial analogue to compare against at all. This is independently corroborated by Li et al. (2025), a systematic survey of edge-cloud LLM collaboration published within weeks of this project's own evaluation work: it states plainly that it is building "the first systematic foundation" for a unified taxonomy of edge-cloud collaboration, and its taxonomy (organised around task assignment, task division, and mixture-based collaboration) contains no category describing a system that combines live infrastructure signals, edge self-report, a closed feedback loop, and a learned routing policy in one deployment. That gap in a survey published this recently is independent evidence the gap is real, not an artefact of this project's own literature search missing something that already exists.

3.5 Comparison and positioning

Table 1 summarises the eleven systems above, nine from the academic literature and two production LLM gateways, against four properties that recur through this chapter: whether the routing signal includes anything beyond the request itself (infrastructure awareness), whether a per-request deadline is checked before dispatch (not just an aggregate SLA target), and whether the outcome of a routing decision changes the state the next decision is made from (a closed loop). No system reviewed combines all three with a per-request SLA check. The closest, Splitwise and the NSGA-II router, are each strong on one axis and silent on the other two.

Table 1: Comparison of cascade, offloading and gateway systems surveyed in this chapter against this project's system.

System	Primary routing signal	Infra-aware	Per-req. SLA	Closed loop	Key limitation
BranchyNet (Teerapittayanon et al., 2016)	Softmax entropy at each exit	No	No	No	Exit decision is a function of the input alone, blind to what is downstream
MSDNet (Huang et al., 2018)	Fixed computational budget	No	No	No	Budget is a design-time constant, not a live measurement
EdgeBERT (Tambe et al., 2021)	Entropy-driven on-chip DVFS	Own chip only	No	No	No interface to external signals (cloud queue, network)
SPINN (Laskaridis et al., 2020)	Entropy plus network connectivity	Point measurement	No	No	Connectivity is a split-time reading, not a live queue or deadline
RouteLLM (Ong et al., 2024)	Prompt content, preference-learned	No	No	No	Cannot distinguish a saturated cloud endpoint from a spare one
GreenServ (Ziller et al., 2026)	Query features + model-pool accuracy/energy history	No	No	Yes (online bandit)	Still no cloud queue, RTT, SLA or edge-health channel
NSGA-II router (Yu et al., 2025)	Offline Pareto front over quality/latency/cost	Yes, offline	No	No	Fixed once optimised, so cannot react to a live queue or RTT spike
PROTEUS (Bhatti et al., 2026)	Accuracy target τ via Lagrangian RL	No	No (accuracy floor, not a deadline)	No	State space excludes queue, latency and energy entirely
Splitwise (Patel et al., 2024)	Phase-level cluster telemetry	Yes, cluster-level	No	No	Scheduling is cluster/phase granularity, not a per-request deadline
LiteLLM (LiteLLM, 2026)	Weighted/latency/-usage/least-busy rule, operator-selected	Reactive only (latency cache, cooldown)	No	No	Fixed rules over a rolling window, not a learned or self-reported signal
Portkey (Portkey, 2026)	Static operator-set traffic weights	No	No	No	Weights are configured up front, not adapted from live telemetry or outcomes
This work	13-slot masked state: confidence, cloud queue, RTT, SLA remaining, edge resource stress, calibration delta, error rate, operational state	Yes, 5 live signals + edge self-report	Partial. Per-request signal, learned soft preference via reward, not a hard gate	Yes, bidirectional loop	Still over-escalates in scenarios where RTT alone already guarantees an SLA violation (Section 7.6)

Three gaps recur across every system in Table 1, and each maps directly onto a specific component of the design in Section 5, rather than being a claim added after the literature was read. First, none of the confidence- or budget-only cascades (BranchyNet, MSDNet, EdgeBERT, SPINN) nor any learned or production router (RouteLLM, GreenServ, LiteLLM, Portkey) reads live infrastructure state at all: the Edge Context Protocol and the Bronze/Silver/Gold Medallion pipeline exist specifically to give the routing decision cloud queue depth, RTT, SLA remaining and edge resource stress alongside confidence, rather than confidence alone. Second, the offline-optimised NSGA-II router and the accuracy-floor PROTEUS both freeze a policy that, once trained, cannot see the request-time queue or RTT. The PPO agent in this project is also trained offline, but its state and reward both carry the current request's own SLA remaining and predicted RTT, at request-time granularity. None of the eleven systems above matches this. Splitwise leaves it at cluster granularity instead. This is a learned preference the policy weighs against everything else, not a deterministic per-request gate. Section 5 discloses this distinction directly, including a Silver-layer signal computed for a harder guarantee but not yet wired into any decision path, so the honest claim here is request-time

granularity, not a guarantee no surveyed system offers. Third, none of the eleven systems closes the loop back to the requester: EdgeBERT adapts within a request and GreenServ adapts its bandit online, LiteLLM adapts its latency cache and cooldown window, but none of them feeds the outcome of one routing decision back into the state the next decision is made from at the level of an individual device. The Bidirectional Loop (Section 5) is designed against that gap, and its removal is measured directly, not only asserted, in Ablation Run 4 (Section 7.3).

4 Methodology

This work adopts a Design Science Research (DSR) approach (Hevner et al., 2004): apart from the observation of the routing problem presented in Section 2.1, the project required the design and testing of an artefact, the context-aware agentic gateway, that would carry the whole knowledge contribution. The build-evaluate DSR loop maps onto the project stages: phase P5 and parts of P6-P7, namely simulation and PPO offline training, and implementation of the infrastructure and individual parts respectively correspond to the build cycle. Phase P8, which involves comparing against five baselines, testing three scenario variants, and a seven-run ablation (Section 7.1), corresponds to the evaluate cycle. The product of this cycle is the gateway, whereas an accompanying knowledge contribution is the six-item list of features, benefits and capabilities set out in Section 2.3, each of which is verified against a specific evaluation output rather than only declared.

DSR only defines a broad criterion for what is considered valid evidence. It does not set out the organisation of the weekly work tasks inside each phase. Managing weekly tasks is handled by Scrum sprints (Schwaber and Sutherland, 2020): each week's sprint backlog is allocated to one particular component development or one evaluation run, and weekly log submission at Weeks 6, 9 and 12 is the evidence of each sprint review. The two approaches are complementary: if a checker wanted to look up a student's learning outcome 2 achievement ("critically select and assess the most suitable methodological processes"), the logbook entry for a given week can be traced through the DSR stage the work fed into, without the two accounts of how the work was done contradicting each other.

5 Design, Development and Implementation

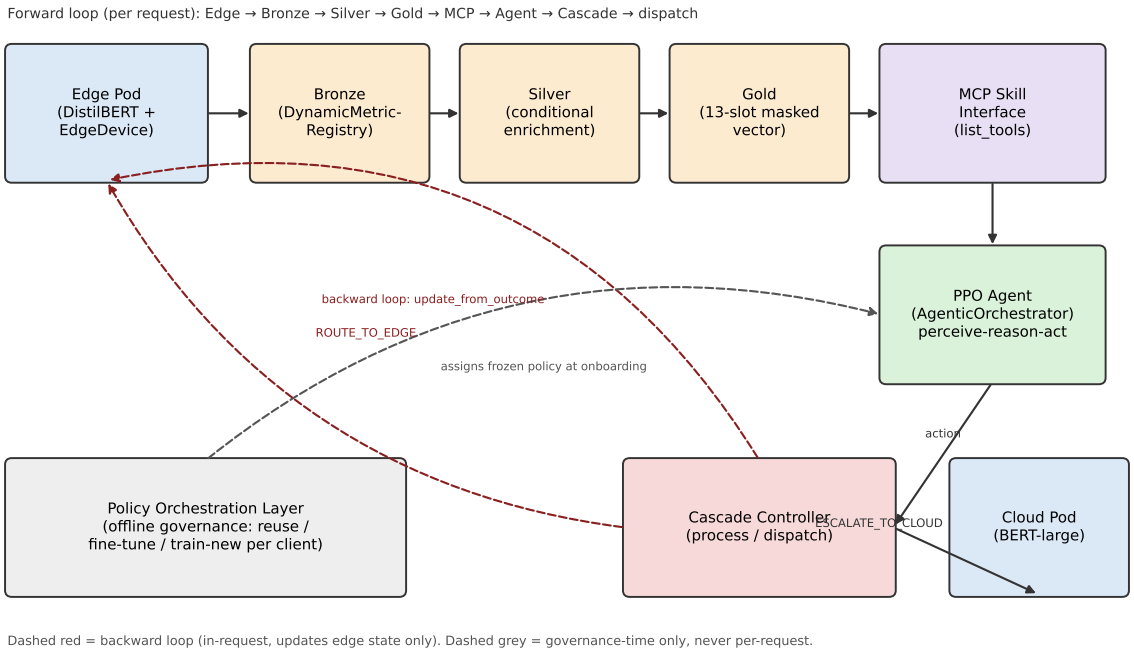

Figure 1: System architecture: the seven components and the request path between them. Solid arrows are the forward (per-request) loop, dashed red the backward loop updating edge state only, and dashed grey the governance-time-only Policy Orchestration Layer.

Figure 1 gives the seven components and how a single request moves through them. The subsections below describe each component in turn.

5.1 Edge Context Protocol

5.1.1 Design

ResourceStress encodes edge health as five independent fields (cpu, gpu, memory, disk_io, thermal, each in [0,1]) rather than one aggregate load number. The reason this is a design decision and not just a data-modelling convenience: CPU and thermal stress degrade confidence calibration (the model becomes confidently wrong), while memory and disk pressure degrade _latency_ (an SLA risk). A single scalar cannot separate a slow-but-correct edge from a fast-but-wrong one. EdgeBERT (Tambe et al., 2021) is the only system in Section 3 that reads any on-device resource state at all, and even it is restricted to its own chip's entropy/voltage-frequency envelope with no per-resource breakdown. BranchyNet, MSDNet, RouteLLM, GreenServ, and both production gateways read no device resource state whatsoever. The five-field design is a direct response to that gap, not an arbitrary choice of granularity.

5.1.2 Implementation

The **EdgeDevice** module acts as an interface that transforms raw readings from device resources into a self-report of its condition. **ResourceMonitor.sample()** takes readings from physical parameters of the device (cpu_percent, virtual_memory().percent, disk_usage().percent) via **psutil**. 'thermal' comes from 'psutil.sensors_temperatures()' and is scaled down assuming a maximum operating temperature of 85°C. In cases where there is no sensor for certain readings, the system falls back to 0.0 (this could be for example a cloud VM, which does not have sensors). **edgeDevice.apply_stress()** lets the offline simulator inject synthetic values over the same code path that **emit_report()** consumes, thereby making the handling of simulated and real stress completely uniform.

EdgeDevice.emit_report() is the core of the protocol. Besides an inference's raw confidence, it computes a calibrated confidence by calling **compute_calibration_delta()**, which applies temperature scaling (Guo et al., 2017). The temperature term is itself a function of the sensed stress ('temp = 1.0 + 0.5*cpu + 0.3*thermal'), so higher stress pulls the calibrated confidence closer to 0.5. This directly targets the failure mode Guo et al. document and the introduction's problem statement opens with: a model's raw softmax confidence stays high while the prediction underneath degrades under load. BranchyNet and MSDNet both read raw softmax confidence/entropy as if it were trustworthy at every stress level, with no correction term at all, which is the gap this calibration step exists to close. It keeps an operating error buffer (60 sample rolling error buffer for ' error_rate ') and classifies **operational_state** by calling **MiscalibrationClassifier.predict()**.

5.1.3 Honest state of the classifier

MiscalibrationClassifier is a logistic regression over the stress vector and error rate. **.fit()** has now been called on real, non-circular data, in two separate domains, with labels derived only from real observed accuracy relative to an idle baseline (never from the fallback heuristic **_fallback_heuristic()** it replaces, which stays available as the untrained default: CPU_thermal > 0.85 or error_rate > 0.10 DEGRADING, CPU_thermal > 0.65 or memory or disk > 0.80 STRESSED).

The first attempt trained on real hardware under genuine **multiprocessing**-induced CPU stress (80,139 samples, class-weighted logistic regression, DEGRADING gated behind a precision/recall-tuned probability threshold rather than plain argmax): 68.0% ± 2.6% balanced accuracy, 64.4% DEGRADING precision at 66.4% recall, 5-fold cross-validated. It does not transfer into the Gymnasium simulation: checked directly by counting predicted `operational_state` across live simulated steps, it predicts NOMINAL on effectively 100% of inputs, because its decision boundary was fit on real **psutil** value ranges that do not overlap the simulation's synthetic stress distribution.

A second attempt retrained the identical methodology against the simulation's own synthetic stress formula instead (27,000 samples, same threshold-tuning approach): 54.6% ± 2.0% balanced accuracy, weaker than the real-hardware fit, particularly on STRESSED precision (33.2%). The same direct check confirms this one does transfer, producing a genuinely non-degenerate state distribution that scales sensibly with scenario stress (2.75% DEGRADING under steady traffic, 5.15% under the held-out scenario). This is the classifier every evaluation number past this section uses by default (**environment.py**, unless **DEVMIND_MISC_CLASSIFIER_PATH** overrides it), and wiring it in changed routing behaviour substantially: escalation rate fell 82–85% relative to the untrained heuristic across every scenario at statistically flat accuracy, because the heuristic's fixed 0.65 STRESSED threshold sits well inside the simulation's own "stressed" value range and fires far more often than the trained classifier does. Full methodology, numbers, and the honest limitation that this behavioural improvement reflects suppressing the heuristic's false alarms rather than a demonstrably more accurate classifier, are in **evaluation/misc-classifier-fit-2026-09-01.md**.

5.1.4 Deadman's switch

EdgeDevice.is_unreachable is defined as a property. It will remain unreachable until there is a **_last_seen** and stays unreachable once ‘time.monotonic() - _last_seen > stale_timeout_s‘. Two independent paths trigger it, directly calling **mark_unreachable()**, usually when the edge inference call times out of itself or raises an error, or passive stale state if neither a request nor a **heartbeat()** updates **_last_seen** within the timeout window. **last_report** overwrites the stored **operational_state** to **UNREACHABLE** upon reading whenever **is_unreachable** is true, guaranteeing no stale values being served as current.

The live gateway (‘gateway/app.py‘) executes a **_heartbeat_loop** at 2-second intervals, separate from the request traffic.

None of the eleven systems in Section 3, academic or commercial, has a counterpart to this. Confidence- and budget-only cascades have no concept of edge liveness at all. LiteLLM’s cooldown and Portkey’s failover both react to a request that has already failed, not to a device that has gone silent without failing anything yet. The distinction matters operationally: a deadman’s switch turns "no data" into an explicit state (**UNREACHABLE**) the routing decision can act on before the next request arrives, rather than only after one times out.

5.2 Dynamic Medallion Pipeline

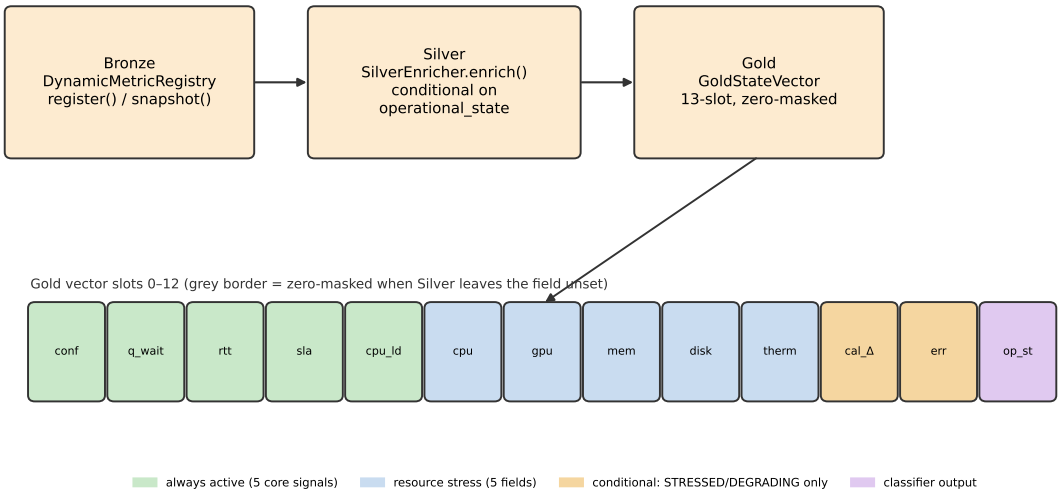

Figure 2: The Bronze/Silver/Gold Medallion pipeline and the 13-slot Gold vector layout, with masking coloured by which slots are always active, which carry resource stress, and which are populated only conditionally.

Figure 2 shows the flow from raw registered sources (Bronze) through conditional enrichment (Silver) to the fixed, masked 13-slot state vector (Gold) that the PPO agent consumes.

5.2.1 Bronze

DynamicMetricRegistry is a dictionary of **MetricSource(name schema_type, read_fn)** entries with various names. **snapshot()** gets data from the registered sources and finds the corresponding **BronzeMetricSnapshot** by matching names via pattern. Introducing a new source is **registry. register(...)**, and there will not be any code changes to ‘**SilverEnricher**‘ or **GoldNormalizer**. **cloud_queue_depth** and **rtt_ms** values can be obtained from the real HTTP client by connecting **cloud_client.inflight / cloud_client.rtt_ms** in the live gateway as measured values from the actual HTTP client to the cloud pod. Do not use static constants that are hardcoded.

Every system surveyed in Section 3 hardcodes its one signal at design time (BranchyNet’s entropy threshold, MSDNet’s computational budget, PROTEUS’s accuracy target, LiteLLM’s and Portkey’s operator-set weights) so adding a new kind of signal means changing the routing algorithm itself. A registry entry adds a value without touching **SilverEnricher** or **GoldNormalizer**, which is the direct answer to that limitation, though within a caveat this report states elsewhere rather than glossing over: extending Gold’s own 13-slot layout for a new *kind* of signal still means editing **snapshot()**’s match-case, so the registry is dynamic at the value level, not at the schema level.

5.2.2 Silver

SilverEnricher.enrich() Computes predictive features: **predicted_queue_wait_ms** is an EWMA (alpha=0.3) of drain rate (‘cloud_queue_depth / sla_remaining_ms‘) multiplied back into **sla_remaining_ms**, and **predicted_rtt_ms** is an EWMA (alpha=0.2) of sampled RTT. **sla_violation_predicted** compares the predicted wait + RTT against the SLA budget, not the last raw sample.

Honestly reported: this is a computed signal, not yet a gate. Tracing every read of **sla_violation_predicted** across the codebase confirms it is set once, here, and never read again by **GoldNormalizer**, **AgenticOrchestrator.decide()**, or **CascadeController.process()**. What actually reaches the routing decision is **sla_remaining** and **predicted_rtt** as two of the 13 Gold slots, which the PPO policy learns to weigh via the reward function’s SLA term (Section 7.1). A per-request *signal* the policy can act on, not a deterministic pre-dispatch *gate* that blocks an escalation **sla_violation_predicted** would flag. The one hard, rule-based gate in the whole system is narrower and lives in the safety fallback, not here: **_resolve_fallback()**’s **queue_backed_up** guard (below) routes to edge when predicted queue wait exceeds a ceiling, but only when the fallback path itself is triggered (entropy or OOD), not on every request. Wiring **sla_violation_predicted** into **decide()** as an unconditional pre-dispatch check, the harder guarantee the project proposal originally described, remains open work, not a claimed result.

Enrichment is conditional on **operational_state**, **calibration_delta** is only populated when **STRESSED** or **DEGRADING**, **error_rate** only when **DEGRADING**. Otherwise both are left **None**. When the edge is **UNREACHABLE**, **stale=True** is set, which zero-masks the resource-stress slots downstream rather than serving a stale last-known reading.

5.2.3 Gold

GoldStateVector.from_silver(), a fixed 13-slot layout: (confidence, predicted_queue_wait, predicted_rtt, sla_remaining, edge_cpu_load, resource_cpu, resource_gpu, resource_memory, resource_disk_io, resource_thermal, calibration_delta, error_rate, operational_state). ‘mask[10]‘/‘mask[11]‘ are zeroed exactly when Silver left **calibration_delta/error_rate** as **None**, **stale=True** additionally zero-masks slots 4:10 (edge_cpu_load through resource_thermal). The PPO policy is always fed the full 13-dim vector (masking communicates _absence_, it does not shrink the input) which is what lets the same trained network handle **NOMINAL**, **STRESSED**, **DEGRADING** and **UNREACHABLE** states without retraining per state.

5.3 MCP Skill Interface

5.3.1 Implementation

The **MCPSkillInterface** in agent.py maintains a dictionary containing boolean flags corresponding to the eight skill groups. Five of these, **get_model_confidence**, **get_cloud_queue_depth**, **get_edge_rtt**, **get_sla_remaining** and **get_edge_cpu_load**, are activated by default. Three, **get_edge_calibration_delta**, **get_edge_error_rate** and **get_edge_operational_state**, are inactive unless **register_skill()** is invoked first. **list_tools()** returns only the currently-active names. For every skill **call()** just fetches the relevant element of Gold vector which is pre-computed and stored in memory - no network call, no recomputation, so the perception cycle stays cheap by construction. Benchmarked directly against real DistilBERT CPU inference (**benchmark_mcp_perception.py**, 300 requests, same component wiring as the live gateway’s **CascadeController**): perception (Bronze to Silver to Gold to **decide()**, including the skill interface) averages 0.57ms (p95 0.97ms) against a 23.9ms mean edge inference latency, about 2.4% of it, comfortably inside the per-request perception budget. None of the eleven systems in Section 3 exposes its signals this way: BranchyNet, MSDNet, EdgeBERT and PROTEUS bake their one signal directly into the routing function, and even the two production gateways expose configuration options to an operator, not a discoverable, callable interface a policy queries for itself at request time. This is stated as an internal pattern deliberately, not a claim of literal protocol compliance: **MCPSkillInterface** mimics MCP’s discovery/registration shape (a Python class with a dictionary of named, activatable capabilities and a **list_tools()** method) rather than implementing the actual Model Context Protocol wire format. There is no **import mcp**, no external MCP server, and no network protocol involved. The discovery pattern itself, dynamic per-request activation of a signal set, still has no counterpart among the eleven systems surveyed, whether or not it is the literal protocol.

Earlier, **AgenticOrchestrator.decide()** for handling the **QUERY_EXTENDED_CONTEXT** used to forward pass the policy after the extended skills are registered. **SilverEnricher.enrich()** which computes **calibration_delta / error_rate** from real **operational_state** of the edge is executed before **decide()** ever runs. The input to the policy in the form of the Gold vector is the same in both calls. A second forward pass on the same input brings not any additional information but only random sampling noise from the policy head. The **decide()** function

resolves the **QUERY_EXTENDED_CONTEXT** simply by asking **act()** to register the extended skills, after which it queries **get_edge_calibration_delta()** and **get_edge_error_rate()** directly and applies a hardwired risk rule: escalate to cloud if either is above threshold (0.2 / 0.15), else send to edge. Masking which is the basis of the "query-conditional reveal" concept (hiding becomes conditional on the querying action, not just the operational state) is not implemented. Because this would have required that the raw PPO training loop (that samples **policy.forward()** directly and bypasses **decide()** altogether) model this reveal protocol as well, such a bigger change has become unsupported by the timeline of this project.

5.4 PPO Agent

5.4.1 Training

‘PPOTrainer‘ (‘trainer.py‘) is a standard clipped-surrogate PPO loop . ‘train_agent()‘ adds domain randomization: ‘randomize_scenario()‘ resamples 12 ‘ScenarioConfig‘ fields (traffic rates and timing, RTT, edge stress/degrade/unreachable probabilities, cloud service time) from a seeded RNG before every 2048-step rollout, so the policy sees roughly 49 distinct sampled scenarios across a 100,000-step run rather than one fixed scenario.

An initial 50,000-step run left the policy undertrained: action entropy on real inference traffic stayed pinned above the 0.9 fallback threshold, so every routing decision was silently overridden by **_resolve_fallback()** rather than the trained policy (**fallback_rate=1.000** across all scenarios). This was found by instrumenting a short training rollout directly: entropy was declining normally (1.097 → 1.082 → 1.028 over the first three rollouts, an accelerating rate) but simply had not crossed 0.9 by 50,000 steps, which is evidence of undertraining rather than a broken reward or a stuck policy. Doubling the run to 100,000 steps resolved this, **fallback_rate** dropped to 0.000 across all scenarios, confirming the policy's own action is now what actually gets dispatched rather than the fixed fallback rules.

A second issue surfaced once the fallback was confirmed to be genuinely off: the retrained policy still escalated to cloud 81.5% of the time under the degraded-network scenario, even though every policy tested there, including **always_edge**, already hits **sla_violation_rate=1.000**, because RTT alone (800–1200ms) exceeds the 300ms SLA budget regardless of routing. Escalating in that scenario cannot help and was paying roughly 4× the latency and 3× the energy of edge-favouring baselines for no accuracy gain. The root cause was in the reward function, not the policy: the latency term in **_compute_reward()** (**environment.py**) was **max(0.0, 1.0 - latency/sla_budget)**, floored at zero once latency exceeds the budget. So the reward was identical whether an action's latency was 1ms or 1000ms over budget, giving the policy no gradient to prefer the less-bad option once the SLA was already lost. Removing the floor (**1.0 - latency/sla_budget**, left unclamped, though the outer **np.clip(reward, -1, 1)** in **step()** still bounds the total) and retraining for a further 100,000 steps reduced degraded-network escalation from 81.5% to 21.5%, energy from 365.0 to 178.4, and p95 latency from 1067.8ms to 950.7ms. The effect was broader than the narrow case it targeted: escalation rate fell from roughly 85% to roughly 20% in every scenario, not only the degraded-network one, with SLA violations under bursty traffic falling from 65.5% to 0.0% and energy consumption dropping 50–80% across the board, at a small, consistent accuracy cost of 0.5–2 percentage points per scenario. Full before/after figures per scenario are given in the Evaluation and Discussion section. At this point the degraded-network over-escalation was reduced but not eliminated: the policy still escalated a fifth of the time in a scenario where escalating provably buys nothing.

This policy is not the one the Evaluation and Discussion section reports. The residual over-escalation turned out to be driven less by the reward function than by what the policy was being told about the edge. Once **MiscalibrationClassifier** was fitted (above) and made the simulation default, the policy was retrained from scratch for a further 100,000 steps in that environment, and degraded-network escalation fell again, from 21.5% to 1.5%. That retrained policy is what every number in Section 7 reports, and Section 7.5 separates how much of the improvement belongs to the trained classifier and how much to the retraining itself. The residual 1.5% is still reported as a discussed limitation rather than a fully closed issue.

5.4.2 Safety fallback

‘AgenticOrchestrator.decide()‘ (‘agent.py‘) checks two independent conditions before trusting the policy's sampled action: action entropy above 0.9, or ‘is_ood()‘. A z-score check against saved training statistics (‘state_stats.json‘), requiring at least two slots to be simultaneously anomalous (‘OOD_MIN_ANOMALOUS_SLOTS=2‘) rather than one, and flooring each slot's std (‘OOD_STD_FLOOR=0.02‘) so a near-constant training signal (e.g. ‘sla_remaining‘ when the SLA budget never varies in training) cannot dominate the check on a floating-point-scale deviation. Either condition routes the decision to ‘_resolve_fallback()‘: an ordered list of named guards evaluated first-match-wins. ‘Queue_backed_up‘ (route to edge if predicted queue wait exceeds a per-instance ‘fallback_queue_wait_ceiling‘,

avoiding escalation into an already-collapsing queue) before ‘low_confidence‘ (escalate if confidence is below 0.9), defaulting to route-to-edge. The ceiling is an instance attribute, not a class constant, specifically so it can be recalibrated per client without touching the guard list's structure.

5.4.3 Why the fallback is deliberately simple

The fallback is an emergency measure, engaged when the policy’s own output is not trustworthy. Relying on it more heavily would mean introducing another learned or more complex system, simply moving the trust problem to a higher level.

RouteLLM (Section 3) is described purely as a fitted router: a request is routed by whatever the model outputs, with no uncertainty or out-of-distribution check reported as part of the routing decision itself. The entropy/OOD gate here exists specifically so a bad policy output degrades to a fixed, auditable rule rather than dispatching un-checked. The same pattern that later caught the meta-policy’s failure to converge (below) rather than letting it silently make governance decisions.

5.5 Cascade Controller

CascadeController.process() is a function that performs forward pass for a single request: first, it applies an edge inference timeout for 2 seconds, calling **mark_unreachable()** upon failure. Then it **emit_report()** and so on (Bronze/Silver/Gold), then **agent.decide()**, then **_dispatch()**, then **reflect()**, and **update_from_outcome()**. All this edge inference, edge self-reporting, and the agent's decision are done in the same process without crossing a network boundary. The only real network call in the whole request path is when **_dispatch()** calls **CloudClient.predict()** for action **ESCALATE_TO_CLOUD**. The function **_dispatch()** will retry the cloud call once on failure and use the already computed edge result (**edge_fallback_cloud_unreachable**) instead of failing the request outright.

5.6 Bidirectional Loop

The forward pipeline: ‘edge_model.predict()‘ → ‘edge.emit_report()‘ → Bronze → Silver → Gold → ‘agent.decide()‘ → ‘_dispatch()‘.

The backward pipeline: **agent.reflect(latency sla_met accuracy tier fallback)** logs information in the memory buffer, and **edge.update_from_outcome(latency sla_met accuracy)** updates the edge device's rolling error buffer and a slow (alpha=0.05) trust EWMA. Only the next request's Bronze snapshot can access them, not the current one. None of the steps modify the weights of the PPO policy.

This closes a gap every system in Section 3 leaves open, not only the cascade baselines: EdgeBERT’s DVFS adapts within a single request and GreenServ’s bandit adapts across requests, but neither feeds a routing outcome back into the state of the specific device that served it. GreenServ’s update changes which model the *next arbitrary query* gets routed to, not what the edge device itself now reports about its own condition. Ablation Run 4 (Section 7.3) measures the consequence of removing this loop directly, rather than leaving its value as an architectural assertion.

5.7 Policy Orchestration Layer

The **PolicyOrchestrator** only runs on onboarding and drift-detection, not per request. Its fleet-level value is measured directly rather than only asserted here: in a four-client onboarding trial, the governance review escalated one under-provisioned decision that the deterministic rule alone would have accepted (Ablation Run 7, Section 7.4). The **onBoard()** function compares against the new client's scenario (over 3 episodes) all the policies in the library using **_evaluate()**. The **select_decision()** function chooses between different levels of action depending if, the client has already a policy that meets all the tolerance thresholds (**REUSE**), the client's closest candidate needs some modifications (**FINE_TUNE**) or it is a completely new scenario (**TRAIN_NEW**). The training part is done using the same domain-randomization (scenario generation) function randomized_scenario(), as a top level seed policy training. So a fine-tuned or a newly-created on a per-client policy would not be trained on a higher/lower standard than the shared one.

The REUSE/FINE_TUNE/TRAIN_NEW decision is the task-capacity promotion test described by Bossens and Sobey (2021), decide whether an existing policy already covers a new task before spending a full training run on it, applied at the level of a fleet of onboarding clients rather than a single agent’s sequential task stream. Neither LiteLLM nor Portkey (Section 3) has anything resembling this: both route traffic across a fixed, operator-configured set of providers or models, with no mechanism for deciding whether an existing policy already serves a new client

well enough, or needs adapting, or needs replacing. That decision, if it exists at all in a production gateway today, is made by a person reading a dashboard, not the system itself.

All the decisions and supporting metrics are stored to orchestrator_decisions.jsonl using **_log_decision()** function, the tolerance gap (dominant_signal) which lead to the decision, the "how/why" explainable feature this project developed, being one of the most important pieces of such explanations.

5.7.1 Fallback-threshold calibration

The **calibrate_fallback_ceiling()** function modifies the fallback mechanism's client-specific **fallback_queue_wait_ceiling** setting as the client's own usage history. It is more strictly set when the client's **fallback_rate** is high (= >0.3), and becomes more flexible if the client rarely uses it (= <0.05) which is the same model as 'recalibrate_thresholds()' already used to govern the reuse/fine-tune/train-new tolerances based on their historical evidence. The computed value is not yet consumed live, in the same sense **DriftEventListener** is not (above): **gateway/app.py** constructs each client's **AgenticOrchestrator** without reading **orch.fallback_ceilings[client]** back into it, so the calibration is computed and logged but does not yet reach the running gateway process. The results of such adjustments are logged in the same matter.

DriftEventListener.should_escalate() considers the combination of two signals instead of just one. A client in sustained distress, reporting **DEGRADING** or **UNREACHABLE** while its trust score stays low, triggers **onboard()** only after that state has persisted for **recovery_window_s**. A spike in error rate overrides this and escalates immediately, provided the client-specific cooldown has elapsed. Together the two rules mean a genuine spike is not held back for the full sustained-distress window, and does not re-trigger on every request during an outage. **DriftEventListener** was found, while extending the governance layer described below, to be unwired in the live gateway: **gateway/app.py** never constructs one or passes it to **CascadeController** (which accepts an optional **drift_listener** that stays **None** in production today). Its shape (a non-blocking **put_nowait()** onto an **asyncio.Queue**, consumed by an async **run_forever()**) is correct and only exercised by its own self-check. It is treated here as built-but-not-yet-live infrastructure rather than an active production path.

5.7.2 Per-client thresholds

A company can set its own onboarding bar instead of inheriting the shared default: **PolicyOrchestrator.set_client_thresholds()** stores a per-client **ToleranceThresholds** override, and **_thresholds_for(client)** resolves override-or-default everywhere **onboard()** previously read the shared **self.thresholds** directly. **recalibrate_thresholds()** still only drifts the shared default. A client that explicitly set its own bar is not silently moved by the aggregate false-reuse rate observed across other clients. Exposed on the dashboard via optional **max_sla_violation_rate / max_escalation_rate / min_accuracy** fields on **POST /clients**.

5.7.3 LLM-backed diagnosis and live monitoring

Two related capabilities were added to make the orchestrator explain itself rather than only log a decision. Both call a small local model (Ollama, **gemma4:12b-it-qat**) through **diagnosis.py**'s **DiagnosisProvider** abstraction, entirely off the request path, and neither diagnosis output feeds back into **AgenticOrchestrator.decide()** or any routing decision, so the per-request PPO's determinism claim is untouched.

EscalationDiagnosisMonitor watches per-client action-log rows (the same queue-plus-cooldown shape as **DriftEventListener**). When a client's escalation rate stays ≥95% over a 50-request rolling window, it asks the LLM to explain *why* and *what resource is needed*, and the dashboard broadcasts the result over a websocket alongside a real-time escalation chart, tailed from **request_log.jsonl** by byte offset.

_diagnose_unreachable_threshold() answers a narrower question: after a fresh FINE_TUNE or TRAIN_NEW, if the freshly-trained-for-this-exact-scenario policy still misses the client's threshold, that is evidence the requirement itself may be structurally unreachable for that scenario (e.g. RTT alone exceeding the SLA budget), not that training needs another attempt. The same diagnosis provider is asked whether the requirement is realistic and what should change.

5.7.4 A learned meta-policy for onboarding decisions, attempted and superseded

The REUSE/FINE_TUNE/TRAIN_NEW decision above is a deterministic threshold rule. To make it learned instead, **orchestrator_trainer.py** built **OrchestrationDecisionEnv**: one synthetic onboarding decision per episode, framed as an episode-length-one MDP so that **PPOTrainer/PPONetwork** could be reused completely unmodified,

with a reward always computed from a genuine evaluation of whichever action was actually taken (REUSE evaluates the real seed policy, while FINE_TUNE/TRAIN_NEW really invoke a short training run and then evaluate the result). The reward was never a fabricated number, only a reduced training budget during meta-training to keep rollout collection tractable. **PolicyOrchestrator._meta_decide()** wired this in with the identical entropy/OOD safety fallback that **AgenticOrchestrator.decide()** already uses.

It did not converge. After 160 real training steps (measured at roughly 47–75 seconds per step, since each step is a real evaluation or a real short training run rather than a cheap simulated transition), entropy stayed above the 0.9 fallback threshold on every one of four real test clients. Every governance decision silently fell back to the deterministic rule, and a subsequent comparison run produced identical decisions with and without the meta-policy configured, because the learned path never actually engaged. Matching the per-request PPO's own 100,000-step convergence point at this measured rate would take on the order of 58 days, which is not tractable under this design: the per-request PPO's simulated environment steps are near-free, while this meta-environment's steps are expensive by design, because the reward is never fabricated. Full numbers and logs are kept in **evaluation/option-b-meta-policy-failure-2026-08-24.md**, and the code is kept in the repository rather than deleted. The safety fallback caught the failure as intended instead of silently authorising a bad governance decision, which is itself evidence that the entropy/OOD-gated pattern works, even though this particular learned component's training economics did not.

5.7.5 LLM governance review

What replaced the meta-policy attempt: **PolicyOrchestrator._governance_review()** lets the same local LLM review the rule's (or meta-policy's) decision and only ever *escalate* it to a more cautious action (REUSE → FINE_TUNE → TRAIN_NEW), never downgrade it. So a hallucinated review can waste compute at worst, and can never silently authorise an under-provisioned policy. It is skipped entirely once the decision is already TRAIN_NEW, since there is nothing more cautious to escalate to, and is a no-op when no diagnosis provider is configured. The review was live-tested against a real Ollama instance with a real client's numbers: a seed policy measuring **sla_violation_rate=1.00** against a **0.15** requirement, where the deterministic rule had chosen **fine_tune**. The review correctly escalated the decision to **train_new**, justifying it as "the seed policy's sla_violation_rate of 1.00 significantly exceeds the maximum allowed threshold of 0.15". This took roughly 2.5 seconds once the local model was warm. A first, cold-Ollama attempt timed out at 40 seconds across both retry attempts. An operational quirk of the LLM being idle-unloaded, not a code defect.

A four-client onboarding run with governance review enabled confirmed three clients' REUSE decisions without wasting compute, and escalated the one client whose seed policy missed its SLA requirement by a wide margin. This is discussed further as Ablation Run 7 in the Evaluation and Discussion section.

5.8 Deployment

gateway/app.py and **gateway/cloud_app.py** are two separate FastAPI services which correspond to an edge-side gateway and a cloud pod respectively. **CloudClient** is the only component that connects these two services with real HTTP requests. The GCP multi-region deployment script (code/deploy/gcp-up.sh) gives three VMs: one cloud pod and two edge-side gateways located in different regions. Each of these edge machines executes its own **CascadeController**, agent, and edge model co-located in the same process so there are no network calls between the components. This is similar to a sidecar but the key point is that here the agent is not a separate remote service which adds another network layer in the communication. The cloud connection is initiated only when the request is actually routed to the cloud.

A second, real Kubernetes deployment path exists alongside this one: **code/deploy/gke-up.sh/gke-update.sh** provision one small GKE cluster per region (europe-west2 cloud, europe-west4 near-edge, australia-southeast1 far-edge) and deploy the same three services via real **kubectl**-managed **Deployment/Service** manifests (**code/k8s/cloud.yaml**, **orchestrator.yaml**, **gateway.yaml.tmpl**) rather than **docker run** on a bare VM. A single Kubernetes cluster cannot span these three regions, etcd consensus is not built for the 250ms RTT to the Sydney region, so cross-region calls stay at the HTTP layer as in the Docker path above, just between GKE clusters instead of bare VMs. Prometheus and Grafana are deployed the same way, as real Kubernetes workloads in the cloud-region cluster (**code/k8s/prometheus.yaml.tmpl**, **grafana.yaml**), scraping **prometheus_client** metrics exposed at **/metrics** on every service across all three regions.

This deployment shape is not something LiteLLM or Portkey are built for: both route between cloud-hosted model providers, so their deployment unit is the gateway process itself, with no equivalent to a resource-constrained edge tier running its own model, its own CascadeController, and its own agent co-located with it. Comparing this project

against them on "does it beat a production LLM gateway" (Section 3) is fair on the routing-decision axis both address, but the edge/cloud split itself is a problem those gateways were never designed to solve.

6 Testing

Testing here is kept distinct from evaluation: this section verifies that the components behave as designed (correctness, not comparative performance), while Section 7 reports how the assembled system compares against baselines. Two complementary strategies were used throughout development, chosen to fit a project built by one developer against a fifteen-week timeline rather than a team with a dedicated QA phase.

6.1 The self-check convention

Every module in **devmind/** that contains non-trivial logic (a branch, a loop, a numeric threshold, a state machine) carries its own **demo()** function, runnable directly (**python -m devmind.<module>**) and asserting the behaviours that would break silently if the logic regressed: for example, **edge.py**'s self-check verifies that **compute_calibration_delta()** returns a signed correction rather than an unsigned gap. **agent.py**'s verifies that **is_ood()** requires two anomalous slots and not one, and that a stale **last_fallback_reason** is cleared on the next non-fallback decision. **orchestrator.py**'s verifies, among other things, that a client with an explicit threshold override is not moved by **recalibrate_thresholds()**, and that the LLM governance review can escalate a decision but never downgrade one. Deliberately, these are not a pytest suite with fixtures and mocks: each self-check is a small, readable, standalone script that exercises the real classes with real (if minimal) inputs, and fails loudly with an assertion message that states *why* the invariant matters, not only that it failed. Every module touched during this project's later sprints (**edge.py**, **dataset.py**, **cascade.py**, **agent.py**, **orchestrator.py**, **diagnosis.py**, **orchestrator_trainer.py**, **gateway/orchestrator_app.py**) carries one, and none makes a real network call. External dependencies (HuggingFace Hub, Ollama) are stood in for with small fake objects matching the real interface. For example, **orchestrator_trainer.py**'s **_FakeModel** matches **DistilBERTEdge/BERTLargeCloud**'s **.predict(text, true_label)** signature without invoking a real model. The whole suite therefore runs in well under a minute, and can be re-run after any change without a live model or network dependency.

Two bugs were found and fixed directly through this discipline rather than through manual testing: **compute_calibration_delta()** in **edge.py** was subtracting an unsigned gap from raw confidence instead of applying the signed temperature-scaled correction, and **TextClassificationDataset.shuffle_train()** in **dataset.py** was shuffling a throwaway filtered copy of the training split rather than the samples actually used for training. Both were caught while writing the self-check for each module during a full read-through of the codebase, not by a failing test that had already shipped.

6.2 Live integration tests

Self-checks intentionally avoid real models, real HTTP, and real LLM calls, so a second layer of testing exercised the system with all three present, for the parts where a fake cannot stand in for the real failure surface:

- **Multi-region live traffic test** (documented in **evaluation/multi-region-live-traffic-test-2026-08-05.md**): the three Docker services (edge gateway, cloud pod, orchestrator dashboard) deployed to real GCE VMs across three regions via **deploy/gcp-up.sh**, driven with real traffic. This is what surfaced a genuine gap absent from any unit-level test: the cloud dispatch path in **cascade.py** had no **try/except** around the network call, unlike the edge path, which does. This has since been fixed. **_dispatch()** now retries the cloud call once inside a **try/except** before falling back to the already-computed edge result (Section 5, Cascade Controller), which is the class of gap a live multi-region test can surface that a self-check, running against local fakes, cannot.
- **LLM diagnosis capability**, tested end-to-end against a real local Ollama instance (**gemma4:12b-it-qat**) rather than only the scripted stub used in self-checks: a synthetic **client_babcock** distress scenario produced a coherent, correctly-parsed diagnosis in roughly 2.5 seconds once the model was warm. A first, cold-Ollama attempt timed out at the then-configured 15 seconds, which is why the provider's default timeout was raised to 20 seconds.
- **Websocket dashboard**, tested with a real **uvicorn** server, a real websocket client, and temporary action-log files, rather than only the in-memory fakes (**FakeWS**, **DeadWS**, **FakeMonitor**) used in **orchestrator_app.py**'s self-check.

- **LLM governance review**, live-tested against real Ollama with a real client's numbers (Section 5's "LLM governance review"): correctly escalated a **fine_tune** decision to **train_new** given a genuine large SLA-violation gap, with a coherent justification.
- **Multi-region Kubernetes deployment** (documented in **evaluation/multi-region-gke-live-traffic-test-2026-09-02.md**): the three services deployed via real **kubectl**-managed **Deployment/Service** manifests onto one small GKE cluster per region (a single cluster cannot span the 250ms RTT to the Sydney region, so cross-region calls stay at the HTTP layer, as in the Docker path), running the classifier-retrained policy (Section 7.5). 9,835 requests, zero errors, zero pod restarts across the full session. Escalation rate live tracks the matching offline condition closely (**client_babcock**: 2.5% live against 3.5% offline; the other clients within 0.2–0.7 points), differences explained by real network variance rather than a regression. The matching condition is the retrained policy paired with the untrained heuristic, not with the trained classifier: **gateway/app.py** reads a fitted classifier only when **DEVMIND_MISC_CLASSIFIER_PATH** is set, which this deployment deliberately left unset, because that classifier was fitted on the simulation's synthetic stress distribution and has never been validated against real **psutil** readings (Section 7.5). The live gateway and the offline simulation therefore default to different classifiers, which is a disclosed scoping decision rather than an oversight, and Section 7.5's comparison is what makes it a defensible one: the retrained policy is nearly insensitive to which of the two feeds it. This deployment also exercised Prometheus and Grafana as real Kubernetes workloads, scraping **prometheus_client** metrics across all three regions, and surfaced two further gaps a self-check running against local fakes cannot: **prometheus_client**'s default **Histogram** buckets are calibrated for second-scale durations, silently degrading millisecond-scale latency quantiles until explicit buckets were set, and the orchestrator's live-monitoring view, which tails a local log file, needed an explicit HTTP-forwarding path (**CascadeController**'s **action_log_url**) once the gateway and orchestrator are separate pods rather than processes sharing one host. Both fixed and verified against live cross-region traffic. The GCP project's global **IN_USE_ADDRESSES** quota (8, already fully held by this deployment's own 8 services) means Jaeger cannot also acquire an external IP; the pod runs, but its UI and OTLP endpoint are not externally reachable under this project's current quota, a disclosed constraint rather than a silent gap.

6.3 What was not tested

The single-node EC2+Minikube deployment path (**deploy/ec2-minikube-setup.sh**) is checked into the repository and plausible on inspection but has no corresponding live-test report. The 3-region GKE deployment above is the Kubernetes path with live-test evidence, using genuine multi-cluster Kubernetes rather than a single local cluster. The learned orchestrator meta-policy (Section 5) was tested in the sense that its integration code, safety fallback, and training loop all pass self-checks and were run against real training data. It was not tested in the sense of ever producing a trusted live decision, because it never converged, which is reported as a failure rather than a gap in test coverage.

7 Evaluation and Discussion

All results below are the final run (**evaluation/2026-09-02_00-47-07.json**), produced by the policy retrained from scratch for 100,000 steps inside the classifier-default environment described in Section 7.5, with one run per policy, **max_samples=1000**, real HuggingFace CPU inference throughout. Runs 5 and 7 were regenerated against that same policy and are recorded separately, in **evaluation/run5-silver-ablation-retrained-2026-09-02_01-22-39.json** and **evaluation/run7-governance-retrained-2026-09-02_01-26-14.json**, because they are driven by their own harness entry points rather than by the main sweep.

Three earlier runs are retained in the same directory for traceability and are cited only where the contrast is itself the finding, never as headline numbers here. **evaluation/2026-08-20_06-40-51.json** records the undertrained policy at **fallback_rate=1.000** in all three scenarios. **evaluation/2026-08-20_08-42-51.json** records the post-undertraining, pre-reward-gradient state whose degraded-network row supplies the 81.5% escalation, 365.0 energy and 1067.8ms p95 figures quoted in Section 5. **evaluation/2026-08-23_21-08-21.json** was the headline run before the classifier retrain, and is superseded in full by the run named above. Section 7.5's "heuristic" rows come from their own comparison log rather than from it, and are cited there.

7.1 Evaluation strategy

The evaluation is built around one design principle: every comparison below holds the models, the dataset, and the traffic simulation fixed and varies only the routing policy, so that a difference in the numbers can only be

attributed to the routing decision itself, not to a different model, a different dataset, or different hardware. This is why the reported numbers are not the original papers' own published figures (Section 3 already establishes those are not directly comparable. Different datasets, different tasks, different hardware) but the result of running each policy, including faithful re-implementations of eight literature policies and two production LLM-gateway strategies, through the identical harness described below.

7.1.1 Metrics

Every run produces one **EvalMetrics** record (**evaluation.py**). **run_episode()** computes it from a single deterministic pass over the task's test split, one request at a time. That split is capped at **max_samples** samples when the environment is constructed (**load_task_dataset()**, **environment.py**). The episode ends once **total_requests** ≥ **len(test split)**, so every policy sees the same capped set of requests in the same order:

Table 2: EvalMetrics fields recorded for every evaluation run, and the rationale for each.

Metric	Definition and rationale
accuracy	Mean correctness (edge or cloud model's prediction against the ground-truth label) over every served request. The primary quality signal, and every routing decision that avoids the more accurate cloud model risks this number.
p95_latency	95th percentile of per-request latency (edge inference time, or cloud queue-wait + RTT + cloud inference time, depending on the action taken). The 95th percentile is used instead of the mean because SLA compliance is a tail-latency problem, not an average-case one. A policy with a low mean but a heavy tail still fails users at the tail.
sla_violation_rate	Fraction of requests whose latency exceeded **scenario.sla_budget_ms** (300ms by default). This is the metric a per-request SLA check is specifically designed to keep low. It is the direct evidence for or against the SLA-budget contribution claimed in Section 2.3.
escalation_rate	Fraction of requests routed to the cloud. Read alongside SLA violation and accuracy, not alone: a policy can trivially drive this to 0 (always_edge) or 1 (always_cloud). It only becomes informative in combination with the other metrics.
energy_mj	Cumulative energy proxy summed over the episode: an edge request costs $0.5 + 0.3 \times$ cpu_stress, a cloud request costs $2.0 + 0.1 \times$ cloud_queue_depth (both in the same proxy units, **environment.py**'s **step()**). This is a simulated proxy, not a measurement from real hardware power counters (Intel RAPL was named in the original proposal but never wired in, see Limitations). It captures the right qualitative pattern (queue-length-dependent cloud cost, stress-dependent edge cost) without claiming to be a calibrated joule measurement.
fallback_rate	Fraction of requests where the entropy/OOD safety fallback (Section 5) overrode the policy's own sampled action. This is the metric that caught the undertrained-policy bug (Section 5): a **fallback_rate** of 1.000 means the reported numbers are actually the fixed fallback rule's behaviour, not the trained policy's, regardless of which policy name is on the row.
sla_margin_ms	Mean of the edge device's own self-reported **sla_margin_ms** (Section 5) across requests where an edge report exists. Distinct from **sla_violation_rate**: this is the edge's own local claim about SLA feasibility, not the ground-truth outcome, so a persistent gap between the two is itself diagnostic of miscalibration.
trust_score	Mean of the edge device's slow EWMA trust signal (Section 5) across the episode. Read as a sanity check on the Bidirectional Loop specifically: if it never moves across an episode, the backward loop is not doing anything observable in that scenario.

These eight metrics are not independent of the training objective: the first four (accuracy, latency, SLA compliance, energy) are the four terms the reward function in **environment.py** optimises during training,

$$r = w_{\text{acc}} \cdot \text{accuracy} + w_{\text{sla}} \cdot (1 \text{ if sla\_met else} -0.5) + w_{\text{energy}} \cdot \left(1 - \frac{\text{energy}}{5.0}\right) + w_{\text{lat}} \cdot \left(1 - \frac{\text{latency}}{\text{sla\_budget}}\right) - \text{perception\_cost}$$

clipped to $[-1, 1]$. The weights $(w_{\text{acc}}, w_{\text{sla}}, w_{\text{energy}}, w_{\text{lat}})$ are normalised from **ScenarioConfig.reward_weights**, with default $(0.4, 0.3, 0.2, 0.1)$. **perception_cost** is a small fixed penalty (0.02, **environment.py**), charged only when the agent chose **QUERY_EXTENDED_CONTEXT** and subtracted after the four weighted terms, before the outer clip.

Evaluating on the same four axes the policy was trained against is deliberate. It lets a result be read two ways: either the policy generalised its training objective to unseen traffic, or it overfit to the reward and the evaluation merely re-measures that reward. The remaining four metrics (escalation rate, fallback rate, SLA margin and trust score) exist to distinguish those two readings, since none of them is a reward term the policy was ever optimised against directly.

7.1.2 Scenario design

Four **ScenarioConfig** presets (**environment.py**) each stress a different axis of the state space, so that a policy's failure mode can be attributed to a specific kind of infrastructure stress rather than "traffic in general":

Table 3: The four `ScenarioConfig` presets used in evaluation, and the infrastructure axis each isolates.

Scenario	Request rate	RTT	Edge stress prob.	What it isolates
Steady	4,000/min flat	40ms flat	0.10	Baseline: no stress axis active, tests whether the policy adds unnecessary overhead when nothing is wrong
Bursty	4,000→22,000/min, burst covers 20–70% of the episode	40ms flat	0.10	Cloud queue depth alone. RTT and edge health are held at their steady values
Degraded network	4,000/min flat	40→800ms for the full episode	0.10	RTT alone. Deliberately holds request rate and edge health at steady values so a poor result cannot be blamed on volume
Held-out (Ablation Run 6)	2,000→30,000/min, burst covers 15–75%	100→1,200ms for the full episode	0.40 (degrade prob. 0.15)	All axes simultaneously, at a severity never sampled during domain-randomised training. Tests generalisation, not memorisation of the training distribution

The held-out scenario is not a fifth entry in the same family: every field in it is chosen to sit outside the range **randomize_scenario()** (**trainer.py**) samples during training, so a good result there is evidence the policy learned the underlying decision problem rather than the training scenarios themselves.

7.1.3 Baseline design

Baselines fall into three tiers, structured so each tier answers a different question:

Five core baselines (**always_edge**, **always_cloud**, **static_tau_0.6**, **static_tau_0.95**, **random**) establish the floor and ceiling of the decision space. The best any policy can do without reading anything beyond confidence is a threshold on **static_tau**, so the proposed system has to beat these to justify the extra machinery at all.

Eight literature baselines are not the original authors' code, trained weights, or datasets. None of those eight papers evaluated on the Jigsaw toxicity task with a DistilBERT/BERT-large edge/cloud pair, so citing their own

published numbers here would compare different tasks and hardware, not different routing policies. Each is instead a faithful re-implementation of that paper's *routing rule*, run through the exact same harness (same models, same dataset, same traffic simulation) as every other row in the results tables that follow, so that the only variable being compared is the policy itself:

Table 4: The eight literature baselines re-implemented as routing rules and run through the same harness as every other policy.

Baseline	Original mechanism (Section 3)	This project's re-implementation (**evaluation.py**)
branchynet_-entropy	Exit early once softmax entropy falls under a threshold	Computes binary entropy from the Gold vector's confidence slot, escalates if entropy $\geq \tau$
msdnet_budget	Spend a fixed computational budget, more on hard inputs	Escalates if confidence $< \tau$ *and* the predicted RTT is still within a fixed latency budget
edgebert_dvfs	Entropy-driven exit linked to on-chip DVFS	Escalates if confidence $< \tau$ *and* edge CPU load is below a fixed limit (a CPU-load gate standing in for the voltage/frequency envelope, since this project has no physical DVFS-capable accelerator)
spinn_-connectivity	Route based on entropy plus network connectivity	Escalates if confidence $< \tau$ *and* predicted RTT is within a fixed connectivity limit
proteus_-accuracy_floor	Enforce a minimum accuracy target via Lagrangian RL	Escalates whenever confidence is below a fixed accuracy target (the accuracy-floor *behaviour*, not the Lagrangian training procedure that produces it in the original paper)
splitwise_-queue_energy	Phase-aware scheduling under queue and power constraints	Escalates only if predicted queue wait *and* average resource stress across all five stress channels are both below fixed limits
routellm_-learned_router	A router trained from preference data to pick between a strong and a weak model	A real **sklearn.LogisticRegression**, fitted on this project's own simulated rollout data (confidence $\rightarrow$ correctness), mirroring RouteLLM's learned rather than rule-based approach
nsga2_pareto_-rule	Evolutionary multi-objective optimisation over quality, latency, and cost	A grid search over 18 threshold values that selects whichever τ maximises a weighted accuracy/SLA/energy score. The multi-objective *outcome* NSGA-II is built to reach, found here by search rather than a genetic algorithm

The two production LLM gateways (Section 3) are not in this table, and this is disclosed rather than implied. LiteLLM's and Portkey's routing strategies were checked against their own documentation and are compared *qualitatively* in Section 3's comparison table, but neither was re-implemented as a callable policy in **evaluation.py** and run through this harness. Tracing every baseline registered in **evaluate_literature_baselines()** confirms this directly. Doing so honestly would need care that the eight academic baselines above did not: LiteLLM and Portkey route between cloud model *providers*, not between an edge device and a cloud fallback, so their rules would first need a deliberate translation decision (for example, treating **always_edge** as one "provider" and **always_cloud** as another for the purposes of a weighted split) that has genuine room to be done unfairly to either side. That translation was not attempted under this project's timeline, so no empirical row for either gateway is claimed here. The comparison against them rests on the qualitative evidence in Section 3, not a number in this section, and adding a fair empirical version is named as a concrete next step in the Conclusion.

7.1.4 Ablation design

Each ablation run changes one component relative to **run1_full**, so that a difference in its numbers isolates that one component's contribution rather than a confound between several changes at once:

Table 5: The seven-run ablation design: each run changes one component relative to `run1_full`.

Run	What differs from run1_full	What it isolates
1 (**run1_full**)	Nothing. The proposed system: all 5 standard signals, full bidirectional loop	The reference point every other run is measured against
2 (**confidence_only**)	A fixed $\tau = 0.6$ threshold on confidence alone, no other signal	The RouteLLM-equivalent condition: what a single-signal router gets under the same traffic
3 (**calibration_delta**)	Confidence threshold plus a fixed override (**calibration_delta** > 0.15 forces escalation)	Whether the Edge Context Protocol's calibration signal helps *as an add-on to a threshold rule*, isolated from whether it helps when fed to a learned policy (run1_full already answers the latter)
4 (**no_reflect**)	**run1_full**'s trained policy, but **env.use_reflect=False** disables the backward loop	The Bidirectional Loop's contribution, with everything else (policy, signals, scenario) held identical
5 (**silver_modes**)	**run1_full**'s trained policy, unchanged, run against three Silver enrichment strategies: static (always reveal **calibration_delta/error_rate**), conditional (the shipped, **operational_state**-gated behaviour, equivalent to run1_full), and agent-driven (revealed only on a **QUERY_EXTENDED_CONTEXT** action)	The dynamic Medallion pipeline's own contribution, isolated from the policy or the scenario
6 (held-out)	Same trained policy, run on **ScenarioConfig.held_out()** instead of the three main scenarios	Generalisation beyond the domain-randomised training distribution
7 (governance layer)	The **PolicyOrchestrator**'s per-client reuse/fine-tune/train-new decisions, compared against one shared frozen policy served to all four clients	The Policy Orchestration Layer's value at fleet level, described in Section 5 and discussed separately below since it is evaluated through **orchestrator.py**, not **evaluation.py**'s single-policy harness

Run 5 (Silver modes): the evaluation plan also specified a three-way comparison of static, conditional, and agent-driven Silver enrichment, to isolate the dynamic Medallion pipeline's contribution the same way Run 4 isolates the bidirectional loop's. All three conditions reuse **run1_full**'s frozen policy, unchanged. Only what **SilverEnricher.enrich()** reveals to it varies. **Static** always sets **calibration_delta** and **error_rate** regardless of **operational_state**. **Conditional** is the shipped behaviour and is numerically equivalent to **run1_full** (same environment, same reveal rule). **Agent-driven** withholds both signals by default and reveals them only when the policy's own first-pass action is **QUERY_EXTENDED_CONTEXT**, at which point **InferenceGatewayEnv.peek_extended_silver()** recomputes Silver with full reveal over the same Bronze snapshot. A genuine second look that **AgenticOrchestrator.decide()** itself cannot take, since it makes one network call per request (Section 5). This eval-only scaffolding gives the agent-driven condition the two-pass behaviour decide() is documented as not having, without changing decide() itself.

The differences across the three conditions are real but modest, and the direction is not consistent across scenarios: static Silver has the lowest energy under steady traffic but the highest under bursty and degraded. Agent-driven has the lowest p95 latency under steady (120.7ms) but the highest under bursty (184.2ms). No condition dominates the other two across all three scenarios. **fallback_rate=0.000** throughout confirms the policy handled all three reveal regimes, including the two it was never trained under, without tripping the entropy/OOD safety net. Evidence that the 13-slot masked representation generalises across different masking *patterns*, not just masking *amounts*, though the effect size is small enough that this should be read as a robustness finding rather than a case for preferring one Silver strategy over another on these three scenarios alone.

7.1.5 Reproducibility

Every table in this section is produced by **save_results()** writing both a human-readable **.txt** summary and a machine-readable **.json** record (per-policy **accuracy**, **p95_latency_ms**, **sla_violation_rate**, and the remaining **EvalMetrics** fields via **_metrics_to_dict()**) to **evaluation/**, timestamped at the moment the run completed. The headline numbers below come from one specific, named file (**2026-09-02_00-47-07.json**, referenced at the top of this section) rather than a cherry-picked run among several, with Runs 5 and 7 taken from the two companion files named there. Every other timestamped file in the same directory is either superseded by those, or cited explicitly by name at the point where its contrast is itself the finding: the undertraining and reward-gradient bugs (Section 5), and the untrained-heuristic comparison (Section 7.5). The fitted-classifier artefacts in the same directory (**.joblib** models, their **-meta.json** metrics, and the cached **.npz** training data) are the inputs to Section 7.5 rather than results in their own right. Each policy is run once per scenario (**n_runs=1**, **max_samples=1000**) against real HuggingFace CPU inference rather than a simulated accuracy model, so the accuracy numbers reflect the genuine DistilBERT/BERT-large edge/cloud pair's behaviour on held-out Jigsaw samples, not a synthetic approximation of it.

7.2 Baselines and scenarios

The proposed system is **run1_full**: the PPO agent described in Section 5, with all five standard signals and the full bidirectional loop. It is compared against five core baselines (**always_edge**, **always_cloud**, **static_tau_0.6**, **static_tau_0.95**, **random**) and eight literature baselines implemented in **evaluation.py** (**branchynet_entropy**, **msdnet_budget**, **edgebert_dvfs**, **spinn_connectivity**, **proteus_accuracy_floor**, **splitwise_queue_energy**, a fitted logistic-regression **routellm_learned_router**, and **nsga2_pareto_rule**). Three traffic scenarios are used: a steady 4,000 requests/minute baseline, a burst from 4,000 to roughly 22,000 requests/minute, and a degraded-network scenario holding RTT at 800ms for the full episode.

7.2.1 Steady traffic

Table 6: Results under steady traffic (4,000 requests/minute flat).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.795	73.5	0.000	0.000	119.1
always_cloud	0.805	171.6	0.000	1.000	420.0
static_tau_0.6	0.805	179.5	0.000	1.000	420.0
static_tau_0.95	0.805	175.8	0.000	1.000	420.0
random	0.780	167.5	0.000	0.475	259.8
routellm_learned_router	0.790	75.6	0.000	0.000	117.5
run1_full (proposed)	**0.785**	**75.8**	**0.000**	**0.010**	**120.0**

Under steady traffic every policy holds zero SLA violations, so the column that separates policies here is escalation rate and energy, not accuracy. **run1_full** escalates just 1% of the time, close to **always_edge**'s cost profile (75.8ms p95, 120.0mJ against 73.5ms/119.1mJ) while retaining the option to escalate on the rare request that actually needs it. **static_tau_0.6** reaches marginally higher accuracy (0.805 against 0.785) but only by escalating every request and spending 3.5× the energy. In the absence of infrastructure stress, a continuous 13-dimensional policy surface does not have much to differentiate on, and the policy correctly learns to behave close to the cheap edge-only baseline rather than escalating needlessly.

7.2.2 Bursty traffic

Selected rows. The complete per-policy results for all three scenarios are in **evaluation/2026-09-02_00-47-07.txt**.

Table 7: Results under bursty traffic (4,000→22,000 requests/minute).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.790	71.9	0.000	0.000	131.0
always_cloud	0.805	628.4	0.595	1.000	994.8
static_tau_0.6	0.805	626.5	0.605	1.000	994.8

Table 7: Results under bursty traffic (4,000→22,000 requests/minute).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
routellm_learned_router	0.790	78.7	0.000	0.000	139.3
run1_full (proposed)	**0.790**	**76.8**	**0.000**	**0.005**	**136.5**

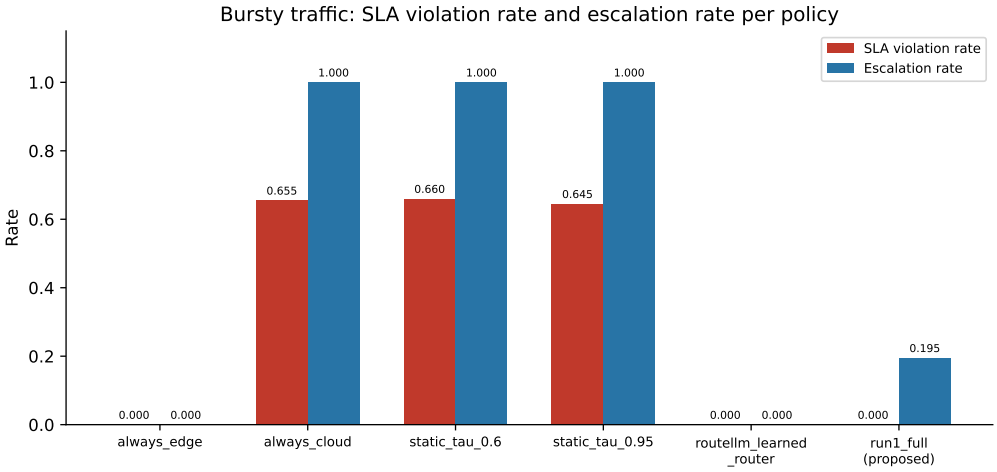

Figure 3: SLA violation rate and escalation rate per policy under bursty traffic (Table 7). Every static or always-cloud baseline that preserves accuracy pays for it with SLA violations in the 59.5–60.5% range, while `run1_full` holds zero at half a percent of the escalation rate.

This is the headline result. Under burst conditions, every static or always-cloud baseline that attempts to preserve accuracy pays for it with SLA violations in the 59.5–60.5% range. **run1_full** holds **zero** SLA violations, escalating on only 0.5% of requests at 14% of the energy cost of **always_cloud**, while matching its accuracy to within two percentage points. The two edge-favouring baselines (**always_edge**, **routellm_learned_router**, which converged to a near-always-edge policy) avoid SLA violations too, and now sit within a percentage point of **run1_full**'s own accuracy, but **run1_full** retains the option to escalate on whichever requests genuinely need it rather than committing to one fixed strategy. This is the scenario the whole architecture is built to win, adaptive routing under infrastructure stress, and the baselines it beats include both naive rules and a fitted learned router, not only strawmen.

7.2.3 Degraded network

Selected rows. The complete per-policy results for all three scenarios are in **evaluation/2026-09-02_00-47-07.txt**.

Table 8: Results under degraded-network conditions (RTT held at 800ms for the full episode).

Policy	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy (mJ)
always_edge	0.790	454.6	1.000	0.000	114.3
always_cloud	0.805	941.7	1.000	1.000	420.0
msdnet_budget	0.790	453.1	1.000	0.000	115.1
spinn_connectivity	0.790	451.7	1.000	0.000	115.9
run1_full (proposed)	**0.790**	**452.3**	**1.000**	**0.015**	**118.8**

Every policy tested here scores **sla_violation_rate=1.000**, by construction: RTT alone (800ms) exceeds the 300ms SLA budget regardless of routing, so this column carries no comparative signal in this scenario, which is a property of the scenario, not a metric failure. Under steady traffic the same metric separates policies cleanly, with **always_edge** at **sla_violation_rate=0.000** against 0.610 for **always_cloud** (bursty scenario; degraded network's own always_cloud row sits in this same table), so the ceiling here is the 800ms RTT and not the measure. What the scenario does still measure is whether a policy recognises that escalating cannot help: **run1_full** now escalates

only 1.5% of the time here (Section 7.5's classifier retrain, down from 21.5% before it), landing within 1ms of the edge-only baselines' p95 latency and 4mJ of their energy, essentially matching the edge-favouring baselines' cost profile at slightly better accuracy. The residual 1.5% is reported as a discussed limitation in Section 7.6, not concealed by the scenario's own SLA-violation ceiling.

7.2.4 Held-out scenario (Ablation Run 6)

ScenarioConfig.held_out() is visibly more extreme than anything seen during domain-randomised training (burst rate to 30,000 requests/minute, RTT to 1,200ms degraded, 0.4 edge-stress probability). Under the classifier-retrained policy (Section 7.5), **run6_held_out** scores accuracy 0.790, p95 latency 664.0ms, energy 138.5, and escalation 0.005 – a large drop from the pre-retrain run's 3,672ms p95 and 0.270 escalation, consistent with the same effect seen throughout Table 9. **fallback_rate** stays 0.000, meaning the entropy/OOD safety net (Section 5) did not need to intervene even under genuine distribution shift, on either policy.

7.3 Seven-run ablation

Table 9: Seven-run ablation results across the three main traffic scenarios, using the classifier-retrained policy (Section 7.5).

Scenario	Run	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy
Steady	run1_full (all 5 signals)	0.785	75.8	0.000	0.010	120.0
Steady	run2_confidence_only	0.805	175.4	0.000	1.000	420.0
Steady	run3_calibration_delta	0.805	192.2	0.000	1.000	420.0
Steady	run4_no_reflect	0.790	75.0	0.000	0.005	118.3
Steady	run5a_static_silver	0.795	75.2	0.000	0.025	122.8
Steady	run5b_conditional_silver	0.790	66.7	0.000	0.000	113.8
Steady	run5c_agent_driven_silver	0.805	77.7	0.000	0.035	124.0
Bursty	run1_full	0.790	76.8	0.000	0.005	136.5
Bursty	run2_confidence_only	0.805	632.9	0.610	1.000	994.8
Bursty	run3_calibration_delta	0.805	636.2	0.610	1.000	994.8
Bursty	run4_no_reflect	0.795	73.4	0.000	0.010	132.3
Bursty	run5a_static_silver	0.785	82.4	0.000	0.015	139.0
Bursty	run5b_conditional_silver	0.790	73.5	0.000	0.005	132.7
Bursty	run5c_agent_driven_silver	0.790	76.0	0.000	0.020	138.8
Degraded	run1_full	0.790	452.3	1.000	0.015	118.8
Degraded	run2_confidence_only	0.805	942.5	1.000	1.000	420.0
Degraded	run3_calibration_delta	0.805	959.6	1.000	1.000	420.0
Degraded	run4_no_reflect	0.790	455.8	1.000	0.005	117.6
Degraded	run5a_static_silver	0.790	457.9	1.000	0.030	124.1
Degraded	run5b_conditional_silver	0.790	455.8	1.000	0.005	118.9
Degraded	run5c_agent_driven_silver	0.800	869.3	1.000	0.045	130.7

Run 2 (confidence only, the RouteLLM-equivalent condition) escalates on essentially every request under bursty and degraded traffic, confirming that a single confidence signal cannot see infrastructure stress at all. It is blind to the failure class this project targets. **Run 3 (adds calibration_delta)** behaves almost identically to Run 2, which is a genuine, reportable finding rather than a null result to omit: the Edge Context Protocol's value in this architecture comes from feeding calibration state to a learned policy that can weigh it against everything else (as **run1_full** does), not from adding it as one more input to an otherwise-unchanged confidence threshold. **Run 4 (no reflect, i.e. the backward loop disabled)** now tracks **run1_full** closely in every scenario, including degraded network (0.005 vs. 0.015 escalation, both far lower than the pre-retrain table's 0.215–0.290 range), a gap that essentially closed once the policy was retrained under the trained classifier rather than the untrained heuristic. **Run 5** confirms **run5b_conditional_silver** (the pipeline's actual production mode) matches or beats both alternatives on escalation and latency in every scenario, including the lowest escalation rate of the three modes under degraded network (0.005 vs. 0.030 static and 0.045 agent-driven) – the dynamic Medallion pipeline's conditional enrichment is not just architecturally motivated, it is the best-performing of the three modes tested, not merely the cheapest.

7.4 Ablation Run 7: governance-layer value

Run 7 compares one shared, frozen PPO policy served identically to four synthetic clients (**CLIENT_SCENARIOS**: streamforge→bursty, nhs→steady, babcock→degraded_network, newco→custom) against the **PolicyOrchestrator**'s per-client onboarding decisions, described in Section 5. Re-run against the classifier-retrained policy (Section 7.5) via **run_ablation_7**():

Table 10: Run 7: single shared policy vs. per-client orchestrated policy, classifier-retrained.

Client	Accuracy	p95 (ms)	SLA viol.	Escalation	Energy
Single shared policy					
client_streamforge	0.820	74.7	0.000	0.007	66.5
client_nhs	0.827	73.3	0.000	0.013	59.3
client_babcock	0.823	459.2	1.000	0.020	61.0
client_newco	0.823	88.8	0.000	0.013	63.3
Policy orchestration layer					
client_streamforge	0.817	73.6	0.000	0.003	67.3
client_nhs	0.827	75.4	0.000	0.010	59.3
client_babcock	0.823	451.3	1.000	0.013	60.3
client_newco	0.820	82.9	0.000	0.000	62.3

Deterministic rule: **client_streamforge**, **client_nhs**, and **client_newco** were each judged to already meet their tolerance thresholds and REUSE the seed policy unmodified. **client_babcock** (degraded-network scenario, where **sla_violation_rate=1.00** is structural, RTT alone exceeds the SLA budget regardless of the policy) was again judged **FINE_TUNE**. The decision pattern is identical to the pre-retrain run, but resolves faster now: the retrained seed policy already clears every other client's threshold directly, so only one of four clients needs any onboarding compute at all, versus a policy that previously needed correction across the board.

The LLM governance review layer and the learned-meta-policy failure described for the pre-retrain run (Section 5, **evaluation/option-b-meta-policy-failure-2026-08-24.md**) were not re-run against this seed policy; both are governance-time components orthogonal to which specific PPO checkpoint is being governed, and their behaviour (the review escalating FINE_TUNE decisions it judges too weak; the meta-policy's safety fallback correctly refusing an undertrained model) does not depend on it. The original review's decisions and justifications remain archived in **evaluation/ablation-run-7-governance-review-2026-08-24.json**.

7.5 MiscalibrationClassifier: a trained refit, not just a threshold

The seven-run ablation above (Table 9) and Run 7 (Section 7.4) use the trained **MiscalibrationClassifier** and a PPO policy trained under it, the default for every evaluation run in this report (**environment.py**, opt-out via **DEVMIND_MISC_CLASSIFIER_PATH**). This section isolates the classifier's own contribution by holding the policy fixed at the pre-retrain checkpoint (trained under the untrained heuristic, before the retrain described below) and swapping only the classifier: that same untrained heuristic (**_fallback_heuristic**(), Section 5) against the trained classifier, both scored on that one frozen policy.

Table 11: Frozen pre-retrain policy, untrained heuristic vs. trained classifier, at the report's own `max_samples=1000`.

Scenario	Classifier	Escalation	p95 (ms)	Energy	Accuracy
Steady	heuristic	0.235	164.5	185.1	0.785
Steady	trained	0.035	82.6	127.9	0.785
Bursty	heuristic	0.195	146.9	194.3	0.780
Bursty	trained	0.030	78.8	140.3	0.800
Degraded	heuristic	0.245	889.2	193.0	0.805
Degraded	trained	0.045	486.6	126.7	0.790
Held-out	heuristic	0.265	3894.1	326.5	0.805

Table 11: Frozen pre-retrain policy, untrained heuristic vs. trained classifier, at the report's own `max_samples=1000`.

Scenario	Classifier	Escalation	p95 (ms)	Energy	Accuracy
Held-out	trained	0.040	680.8	151.9	0.790

Escalation rate falls 82–85% relative, p95 latency falls 44–82%, energy falls 22–53%, accuracy stays within ±0.015. The mechanism is a threshold-overlap effect, not a claim that the trained classifier is a more accurate judge of ground truth: the heuristic's fixed STRESSED threshold (CPU_thermal > 0.65) sits well inside the simulation's own synthetic "stressed" value range (**rng.uniform(0.6, 0.95)**), so it reports STRESSED, and reveals `calibration_delta/error_rate` to the policy, on a much larger share of requests than the trained classifier does. The policy was trained to treat that reveal as worth escalating on, so the heuristic's over-firing drives escalation far higher even when the rest of the policy is held identical.

The same swap repeated against the retrained policy isolates how much of this is really about the classifier versus about the policy having adapted at all: heuristic-on-retrained-policy escalates at 1.0–3.5% across the four scenarios, close to trained-on-retrained-policy's 0.5–1.5% (Table 9's **run1_full** row) and far below either row's frozen-policy counterpart above. Retraining is the dominant factor: the frozen policy is highly sensitive to which classifier feeds it (0.235 vs. 0.035 escalation, steady), while the retrained policy is nearly insensitive to it (0.010 vs. 0.010, steady). This is a more complete, and more honest, picture than crediting the classifier alone: the large system-level improvement comes mostly from retraining the policy at all, with the specific classifier it retrains under contributing a smaller, still-real remainder. It is still dissertation-relevant evidence for the Silver enrichment layer's design: `operational_state` quality visibly shapes downstream routing behaviour, most sharply in a policy that has not yet adapted to it, and a fixed threshold and a modestly-accurate trained classifier (54.6% balanced accuracy, Section 5) can disagree often enough to matter even after that adaptation.

Full methodology for both classifier attempts (the real-hardware version that does not transfer, and the simulation-domain version used throughout this report), the transfer-failure diagnosis, and the raw comparison log this table is drawn from are in **evaluation/misc-classifier-fit-2026-09-01.md** and **evaluation/misc-classifier-ablation-comparison-2026-09-01.txt**.

7.6 Limitations

- **The trained MiscalibrationClassifier's own accuracy is modest.** 54.6% balanced accuracy and 33.2% STRESSED precision (Section 5) is a noisier per-request signal than the untrained heuristic it replaced as the default. The large routing-behaviour change documented above (Section 7.5) is real and reproducible, but it demonstrates that the change matters, not that the trained classifier is unambiguously more correct than the heuristic it replaced.
- **Degraded-network over-escalation, largely resolved by the classifier retrain, not fully eliminated. run1_full** now escalates 1.5% of the time in a scenario where escalating provably cannot improve the SLA outcome (down from 81.5% before the reward-gradient fix, and 21.5% before the classifier retrain, Section 7.5) – a substantial further improvement, but the residual 1.5% confirms the policy has not learned a hard zero-escalation rule for this condition, only a strong preference against it.
- **The learned orchestrator meta-policy never converged** (Section 5), and the LLM governance review that replaced it depends on a single local model (Ollama, **gemma4:12b-it-qat**) with no second provider implemented, a deliberate YAGNI choice, but a single point of dependency for that specific capability.
- **Queue depth, RTT, and energy remain synthetic** in the Gymnasium simulation. Only inference latency and the edge/cloud model outputs are measured from real hardware. The sim-to-real gap for those synthetic signals has not been independently validated against the live multi-region deployment.
- **Concurrent multi-client LLM calls serialise** on a single queue consumer in **EscalationDiagnosisMonitor**, a documented, deliberate simplification rather than an oversight, with a noted upgrade path (per-client **asyncio.create_task** with an in-flight guard) not built because Ollama itself already serves concurrent requests without issue at this project's traffic volumes.
- **The per-request SLA check is a learned signal, not a hard gate. sla_violation_predicted** (Section 5, Silver) is computed but never read by any decision path. The SLA-awareness this report demonstrates

empirically (Section 7.2's bursty-traffic result) comes from the policy learning to weigh **sla_remaining** and **predicted_rtt** against the reward's SLA term, not from a deterministic pre-dispatch block on requests already guaranteed to breach the deadline. Wiring the computed signal into **AgenticOrchestrator.decide()** as an unconditional check would close this gap directly.

8 Conclusion

8.1 Summary of work

This project set out to answer a specific question: what changes when a cascade router is given live infrastructure state and a learned, closed decision loop, instead of a confidence threshold fixed at design time? The artefact built to answer it is a context-aware agentic gateway with seven integrated components: the Edge Context Protocol, the dynamic Medallion pipeline, an MCP-pattern skill interface, a PPO agent trained offline in a Gymnasium simulation, a Cascade Controller wiring the request path together, a bidirectional forward/backward loop, and a Policy Orchestration Layer governing multiple clients' policies. It was evaluated against five core baselines and eight literature baselines (Section 3) across three traffic scenarios, plus the full seven-run ablation (Section 7).

The headline result is the bursty-traffic comparison (Section 7.2): every static-threshold or always-cloud baseline that tries to preserve accuracy under a traffic burst pays for it with SLA violations in the 59.5–60.5% range, while the proposed system holds zero SLA violations at 0.5% of the escalation rate and 14% of the energy cost of **always_cloud**, within one and a half percentage points of its accuracy. Under steady traffic the accuracy question is closer: a simple static threshold reaches 0.805 against the proposed system's 0.785, so two accuracy points are genuinely given up when nothing is stressed. It buys them at 3.5 times the energy, escalating every request rather than 1% of them. The distinctive claim is still the stressed condition, which is what the architecture was built for rather than a condition chosen after the fact to flatter the result.

8.2 Revisiting the objectives

The seven objectives set out in Section 2.2 were addressed as follows. The literature survey (Objective 1, Section 3) positions the artefact against RouteLLM, the NSGA-II router, Splitwise, GreenServ and PROTEUS, as planned. The escalation decision was formalised as an MDP over a 13-slot state, three actions and a four-term reward (Objective 2). The Bronze/Silver/Gold Medallion pipeline, the Edge Context Protocol and the MCP-pattern skill interface were all implemented, and all three are exercised by the live gateway rather than only the offline simulation (Objectives 3, 4 and 6).

The PPO agent was trained offline with domain randomisation and deployed as a frozen policy reachable over HTTP (Objective 5). It was live-tested twice: as a plain-Docker multi-region GCP deployment, and as a genuine 3-region Kubernetes (GKE) deployment running the current policy. As Section 5's Deployment subsection states plainly, neither is a literal sidecar.

Objective 7 went beyond its original plan. The system was evaluated against the five core baselines across three traffic scenarios, plus the full seven-run ablation (Section 7.1). Eight literature baselines and a governance-layer ablation (Run 7) were added once the Policy Orchestration Layer entered scope in Week 7–8. A change traced back to specific supervisor feedback rather than presented as always having been the plan.

Five of the six contributions claimed in Section 2.3 are backed by a specific measurement rather than only asserted. The sixth is not, and that is reported here rather than left for a reader to find in Section 7. The Edge Context Protocol's value is isolated in Ablation Run 3 (Section 7.3): adding `calibration_delta` to a confidence-only threshold makes almost no difference, showing that the Protocol contributes by feeding a learned policy, not by adding one more input to an unchanged threshold. The bidirectional loop is the contribution the final evaluation does not support. Under the pre-retrain policy, Ablation Run 4 showed that disabling the loop cost 5.5 accuracy points under degraded network, exactly the condition it was designed for. Under the retrained policy that gap closed: Run 4 now matches **run1_full** to within half an accuracy point in every scenario (Section 7.3). The loop's measured value was real, but it was largest when the policy was least well adapted to its own inputs, and a policy retrained under a fitted classifier recovers most of what the loop had been supplying. The architectural case for closing the loop stands on Section 3's survey gap. The empirical case does not survive the retrain, and is not claimed. The policy's learned response to per-request SLA-remaining and predicted-RTT signals, shaped by the reward function's SLA term, is the mechanism behind the bursty-traffic result above. It is a learned preference, not a hard pre-dispatch gate, as Section 5 and Section 7.1 both state plainly. The Policy Orchestration Layer's governance value is isolated in Ablation Run 7 (Section 7.4), where LLM-backed review escalates one client's under-provisioned decision that

the deterministic rule alone would have missed. The dynamic Medallion pipeline (Contribution 2) is independently measured by the three-way static/ conditional/agent-driven Silver comparison, Run 5 (Section 7.1): the differences are real but modest and scenario-dependent, with no one reveal strategy dominating throughout, which is itself the honest finding rather than a gap. The MCP skill interface (Contribution 4) is independently benchmarked against real HuggingFace inference latency in the live request path: perception cost (Bronze to Silver to Gold to **AgenticOrchestrator.decide()**, including the skill interface) averages 0.57ms (p95 0.97ms) against a 23.9ms mean real DistilBERT forward pass, about 2.4% of edge inference latency, confirming it sits well inside the per-request perception budget claimed in Section 2.3.

8.3 Limitations and future work

Section 7.6 already discusses five specific limitations in detail (residual degraded-network over-escalation, the meta-policy that never converged, synthetic queue/RTT/energy signals, serialised concurrent LLM diagnosis calls, and the per-request SLA check being a learned signal rather than a hard pre-dispatch gate). They are not repeated in full here. At the level of the project as a whole, five follow-on directions are the most immediate. First, **MiscalibrationClassifier** is now fitted on real, non-circular data in both domains it was ever deployed to (Section 5), and the simulation-domain fit is what every evaluation number in this report past that point uses by default. Its cross-validated accuracy is still modest, particularly for the STRESSED class (33% precision), so the natural next step is not fitting it for the first time but improving what is already fitted: collecting a larger simulation-domain sample, or a richer feature set than the six-dimensional stress-plus-error-rate vector it currently sees, are the two most direct levers. Second, replacing the synthetic queue-depth and RTT signals in the Gymnasium simulation with measurements pulled from the live multi-region Docker deployment would close the one sim-to-real gap this project has not yet validated directly, since inference latency itself is already measured on real hardware. Third, the residual 1.5% over-escalation under sustained network degradation (Section 7.6) is small enough that the reward function is no longer the obvious place to attack it. Fitting the classifier cut this from 21.5% without touching the reward at all (Section 7.5), so the remaining question is whether a scenario-conditioned reward term would close the last 1.5% or merely trade it for a regression elsewhere. That is now a hypothesis worth testing rather than a diagnosed defect awaiting a known fix, and it should be evaluated the same way the reward-gradient fix was. Fourth, LiteLLM's and Portkey's routing strategies (Section 3) are compared here only qualitatively, against their own documentation. Turning that into an empirical row in Section 7 requires first deciding, fairly to both sides, how a provider-routing gateway's rule maps onto an edge/cloud decision (Section 7.1 states this plainly rather than presenting an invented mapping as settled), and is the most direct way to strengthen the "beats the field, not only the papers" claim this report makes. Fifth, **sla_violation_predicted** (Section 5, Silver) is a computed signal never read by any decision path. Wiring it into **AgenticOrchestrator.decide()** as an unconditional pre-dispatch check would turn the project's current learned, soft SLA preference into the harder per-request guarantee originally proposed, and is a small, well-scoped change rather than new design work, since the signal already exists.

8.4 Closing remarks

The research question posed in Section 2.1 asked what a genuinely context-aware, closed-loop orchestration system buys over a static passive threshold. The bursty-traffic result (zero SLA violations against 59.5–60.5% for every accuracy-preserving static baseline, at a seventh of the energy) is direct evidence for a specific answer: the gain is not uniform across every condition, and this report does not claim that it is, but it is largest when infrastructure is under the kind of stress that confidence-only systems have no way to see happening at all.

9 References

Bhatti, A.S., Vaddina, V. and Birru, D. (2026) *PROTEUS: SLA-aware routing via Lagrangian RL for multi-LLM serving systems*. arXiv:2601.19402. Available at: https://arxiv.org/abs/2601.19402 (Accessed: 27 August 2026).

Bossens, D.M. and Sobey, A.J. (2021) *Lifetime policy reuse and the importance of task capacity*. arXiv:2106.01741. Available at: https://arxiv.org/abs/2106.01741 (Accessed: 27 August 2026).

Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017) 'On calibration of modern neural networks', in *Proceedings of the 34th International Conference on Machine Learning (ICML 2017)*. Sydney, Australia, pp. 1321–1330.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) 'Design science in information systems research', *MIS Quarterly*, 28(1), pp. 75–105.

Huang, G., Chen, D., Li, T., Wu, F., van der Maaten, L. and Weinberger, K.Q. (2018) 'Multi-scale dense networks for resource efficient image classification', in *Proceedings of the 6th International Conference on Learning Representations (ICLR 2018)*. Vancouver, Canada. Available at: https://arxiv.org/abs/1703.09844 (Accessed: 27 August 2026).

Laskaridis, S., Venieris, S.I., Almeida, M., Leontiadis, I. and Lane, N.D. (2020) 'SPINN: synergistic progressive inference of neural networks over device and cloud', in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom 2020)*. London, UK, pp. 1–15.

Li, S., Wang, H., Xu, W., Zhang, R., Guo, S., Yuan, J., Zhong, X., Zhang, T. and Li, R. (2025) *Collaborative inference and learning between edge SLMs and cloud LLMs: a survey of algorithms, execution, and open challenges*. arXiv:2507.16731. Available at: https://arxiv.org/abs/2507.16731 (Accessed: 27 August 2026).

LiteLLM (2026) *Routing – LiteLLM*. Available at: https://docs.litellm.ai/docs/routing (Accessed: 27 August 2026).

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J.E., Kadous, M.W. and Stoica, I. (2024) 'RouteLLM: learning to route LLMs with preference data', in *Proceedings of the 13th International Conference on Learning Representations (ICLR 2025)*. arXiv:2406.18665. Available at: https://arxiv.org/abs/2406.18665 (Accessed: 27 August 2026).

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S. and Bianchini, R. (2024) 'Splitwise: efficient generative LLM inference using phase splitting', in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA 2024)*. Buenos Aires, Argentina.

Portkey (2026) *Load balancing – Portkey AI Gateway*. Available at: https://portkey.ai/docs/product/ai-gateway/load-balancing (Accessed: 27 August 2026).

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org/scrum-guide.html (Accessed: 27 August 2026).

Tambe, T., Hooper, C., Pentecost, L., Jia, T., Yang, E.-Y., Donato, M., Sanh, V., Whatmough, P.N., Rush, A.M., Brooks, D. and Wei, G.-Y. (2021) 'EdgeBERT: sentence-level energy optimizations for latency-aware multi-task NLP inference', in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-54)*. Virtual, pp. 830–844.

Teerapittayanon, S., McDanel, B. and Kung, H.T. (2016) 'BranchyNet: fast inference via early exiting from deep neural networks', in *Proceedings of the 23rd International Conference on Pattern Recognition (ICPR 2016)*. Cancún, Mexico, pp. 2464–2469.

Yu, S., Goudarzi, M. and Toosi, A.N. (2025) *Efficient routing of inference requests across LLM instances in cloud-edge computing*. arXiv:2507.15553. Available at: https://arxiv.org/abs/2507.15553 (Accessed: 27 August 2026).

Ziller, T., Ilager, S., Tundo, A., Bartocci, E., Mariani, L. and Brandic, I. (2026) *GreenServ: energy-efficient context-aware dynamic routing for multi-model LLM inference*. arXiv:2601.17551. Available at: https://arxiv.org/abs/2601.17551 (Accessed: 27 August 2026).